ENHANCED MULTI-ZONAL FOREST TYPE CLASSIFICATION USING YOLOv8 AND EUROSAT FOR SCALABLE ENVIRONMENTAL MONITORING

A Project Report submitted in the partial fulfillment of
the Requirements for the award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Nakka Vijay Bhaskar Reddy (22471A05H8)
Mundlamuri Vijaya Kumarachari (22471A06H6)
Nagaram Prasad Rao (22471A05H7)

Under the esteemed guidance of

Dr. Sireesha Moturi, M.Tech., Ph.D.

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET

(AUTONOMOUS)

Accredited by NAAC with A+ Grade and NBA under

Tyre -1 and an ISO 9001:2015 Certified

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada

KOTAPPAKONDA ROAD, YALAMANDA VILLAGE, NARASARAOPET- 522601

2025-2026

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project that is entitled with the name "ENHANCED MULTI-ZONAL FOREST TYPE CLASSIFICATION USING YOLOv8 AND EUROSAT FOR SCALABLE ENVIRONMENTAL MONITORING " is a bonafide work done by Nakka Vijay Bhaskar Reddy (22471A05H8), Mundlamuri Vijaya Kumarachari (22471A06H6) and Nagaram Prasad Rao (22471A05H7) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

PROJECT GUIDE

Dr. Sireesha Moturi, B.Tech., M.Tech., Ph.D.
Associate Professor

PROJECT CO-ORDINATOR

D.Venkata Reddy, B.Tech., M.Tech., (Ph.D).
Assistant Professor

HEAD OF THE DEPARTMENT

Dr. S. N. Tirumala Rao, M.Tech., Ph.D.
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We declare that this project work titled "**ENHANCED MULTI-ZONAL CLASSIFICATION USING YOLOv8 AND EUROSAT FOR SCALABEL ENVIRONMENT MONITORING**" is composed by me that the work contain here is my own except where explicitly stated otherwise in the text and that this work has been not submitted for any other degree or professional qualification except as specified.

N.Vijay Bhaskar Reddy (22471A05H8)
M.Vijaya Kumarachari (22471A05H6)
N.Prasad Rao (22471A05H7)

ACKNOWLEDGEMENT

I wish to express my thanks to various personalities who are responsible for the completion of my project. I am extremely thankful to my beloved chairman, Sri M. V. Koteswara Rao, B.Sc., who took keen interest in me in every effort throughout this course. I owe my sincere gratitude to my beloved principal, Dr. S. Venkateswarlu, Ph.D., for showing his kind attention and valuable guidance throughout the course.

I express my deep-felt gratitude towards Dr. S. N. Tirumala Rao, M.Tech., Ph.D., HOD of the CSE department, and also to my guide, Dr. Moturi Sireesha, B.Tech., M.Tech., Ph.D., Professor of the CSE department, whose valuable guidance and unstinting encouragement enabled me to accomplish my project successfully in time.

I extend my sincere thanks to D. Venkat Reddy, B.Tech., M.Tech., (Ph.D.), Assistant Professor & Project Coordinator of the project, for extending his encouragement. Their profound knowledge and willingness have been a constant source of inspiration for me throughout this project work.

I extend my sincere thanks to all the other teaching and non-teaching staff in the department for their cooperation and encouragement during my B.Tech. degree.

I have no words to acknowledge the warm affection, constant inspiration, and encouragement that I received from my parents.

I affectionately acknowledge the encouragement received from my friends and those who were involved in giving valuable suggestions and clarifying my doubts, which really helped me in successfully completing my project.

By

N. Vijay Bhaskar Reddy (22471A05H8)

M. Vijaya Kumarachari (22471A05H6)

N. Prasad Rao (22471A051H7)

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research

M2: Build a passionate and a determined team of faculty with student centric teaching, imbibing experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENTOFCOMPUTERSCIENCEANDENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertize in modern software tools and technologies to cater to the real time requirements of the Industry.

M3: Inculcate team work and lifelong learning among students with a sense of societal and ethical responsibilities.

Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

Program Educational Objectives (PEO's)

The graduates of the programme are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to the academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

Program Outcomes

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science,computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis&interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering&IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national&international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Project Course Outcomes (CO'S):

CO421.1: Analyse the System of Examinations and identify the problem.
CO421.2: Identify and classify the requirements.
CO421.3: Review the Related Literature
CO421.4: Design and Modularize the project
CO421.5: Construct, Integrate, Test and Implement the Project.
CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓	✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3	3				2		
C421.4			2			1	1	2				3	2	
C421.5					3	3	3	2	3	2	2	3	2	1
C421.6									3	2	1	2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1. Low level
2. Medium level
3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description of the device	Attained PO
CO421.1, C2204.2, C22L3.2	Identified the real-world problem of forest stand detection; gathered requirements; analyzed dataset characteristics (EuroSAT, Sentinel-2 RGB bands)	PO1, PO2, PO8
CC421.1, C2204.3, C22L3.2	Performed extensive literature survey on forest detection, feature selection, CNN models, and YOLO variants; finalized methodology	PO2, PO5, PO8
CC421.3, C22L3.3	Designed the model architecture and w classification system, UML-style logical workflow	PO3, PO5, PO8
CC421.4, C2204.3, C22L3.2	Implemented preprocessing, data norm hyperparameters	PO4, PO11, PO8
CC421.5, C22L3.2	Tested the trained model using accurac matrix analysis and compared YOLOv 8 vs EfficientNet-B4	PO11, PO4, PO8
CC421.5, C2204.2, C22L3.3	Prepared documentation including results, graphs, experimental setup, failure case analysis, conclusions	PO10, PO8
C2202.2, C2204.3, C4110.2	Designed detailed visualization output matrices, confidence-level charts, and performance in real-world forest classi	PO4, PO7, PO8

C32SC4.3,C3204.3	Evaluated environmental and societal significance—forest monitoring, sustainability, ecosystem protection	PO5, PO6, PO8

ABSTRACT

Forests are a critical component of the Earth's ecosystem, yet monitoring them accurately over large areas remains a significant challenge. This research presents an inteligent deep leaning (DL) approach using satellite imagery to achieve high-accuracy forest stand detection. The study aims to develop a faster, scalable, and automated forest analysis framework compared to traditional field-based surveys, utilizing the EuroSAT dataset for effective land cover classification. The YOLOv8-nano model was trained and evaluated to assess its capability in identifying forest and non-forest regions. A total of 27, 000 satellite images across 10 distinct land-cover classes were used for validation, ensuring diverse and comprehensive data coverage. The model achieved a peak accuracy of 97.5%, demonstrating robust performance and real-time detection capability. These findings highlight the strong potential of the YOLOv8 architecture for environmental monitoring, forest conservation, and sustainable resource management. Overall, this study represents an important step toward integrating artificial intelligence and satelite imagery for efficient and scalable forest observation.

Keywords: Forest Type Classification, YOLOv8-nano, EuroSAT Dataset, Sentinel-2, Deep Learning, Remote Sensing, Environmental Monitoring

INDEX

LIST OF FIGURES

1. INTRODUCTION

Forests are among the most critical natural resources on Earth, playing a vital role in maintaining ecological balance, supporting biodiversity, regulating climate, and contributing to sustainable socio-economic development. They act as major carbon sinks by absorbing carbon dioxide from the atmosphere and help mitigate the impacts of climate change. Forest ecosystems also support diverse flora and fauna, protect soil and water resources, and provide livelihoods through timber and non-timber forest products. However, rapid deforestation, illegal logging, forest fires, and climate-induced changes have significantly threatened forest ecosystems worldwide, making continuous and accurate forest monitoring essential for sustainable forest management and environmental conservation **[1]**.

Traditionally, forest monitoring has relied on ground-based field surveys, manual interpretation of aerial photographs, and visual analysis of satellite images. While these approaches provide reliable information at a local scale, they are time-consuming, labor-intensive, costly, and unsuitable for monitoring large or remote forest regions. Manual interpretation also introduces subjectivity and inconsistencies, which affect the reliability of long-term forest assessment **[2]**. These limitations highlight the need for automated and scalable monitoring solutions.

Recent advancements in remote sensing technology have significantly transformed environmental monitoring practices. Earth observation satellites such as Sentinel-2 and Landsat provide high-resolution imagery with frequent temporal coverage, enabling large-scale observation of forest conditions and land-use changes. Satellite-based monitoring allows consistent, repeatable, and wide-area forest analysis, making it a practical alternative to traditional field-based approaches **[5], [9]**. However, the massive volume of satellite data generated daily

makes manual analysis impractical, creating the need for intelligent automated processing techniques. Artificial Intelligence, particularly deep learning, has emerged as a powerful tool for automated image classification and object detection. Convolutional Neural Networks (CNNs) have demonstrated exceptional performance in image analysis tasks by automatically learning complex spatial and spectral features from raw image data, eliminating the need for manual feature engineering. In recent years, CNN-based models have been successfully applied to remote sensing tasks such as land cover classification, crop monitoring, and urban mapping. Among the various architectures, the YOLO (You Only Look Once) family of models is widely recognized for its ability to process images in real-time with high accuracy. The latest version, YOLOv8, extends these capabilities to classification tasks in addition to detection, making it a promising choice for large-scale environmental applications.

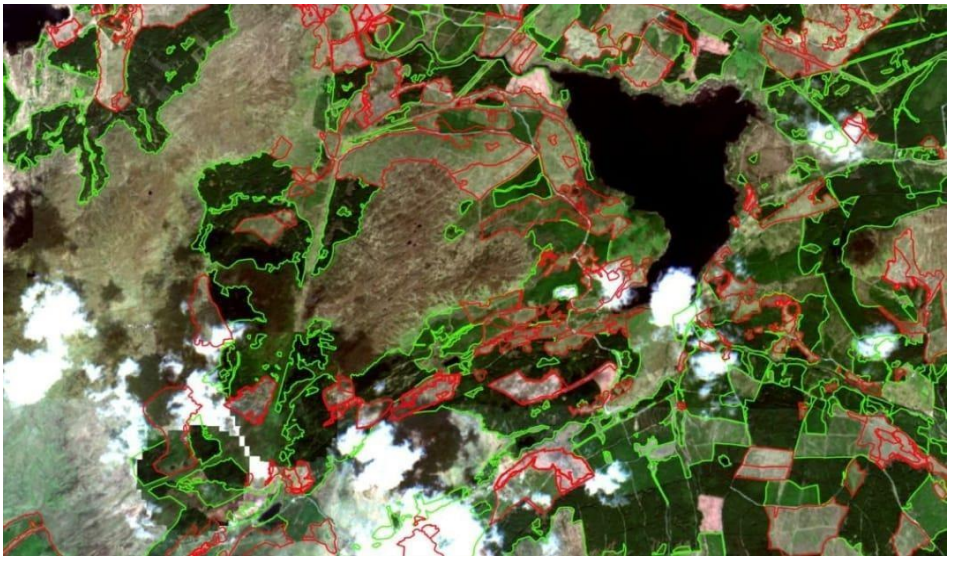

Fig 1. Satellite-Based Forest Monitoring

Fig1:illustrates the concept of satellite-based forest monitoring, where Earth observation satellites capture high-resolution images of forested regions. These images are processed using artificial intelligence and deep learning models to classify forest types, detect land-cover

changes, and monitor environmental conditions. Satellite-based monitoring enables continuous and large-scale assessment of forests, overcoming the spatial and temporal limitations of ground surveys and manual image interpretation **[1]**.

Artificial Intelligence (AI), particularly deep learning, has emerged as a powerful tool for automated satellite image analysis. Convolutional Neural Networks (CNNs) are capable of learning complex spatial and spectral features directly from raw imagery, eliminating the need for handcrafted feature extraction. Deep learning approaches have demonstrated superior performance in land cover classification, forest fire detection, and environmental monitoring compared to traditional machine learning techniques **[3], [4], [8]**.

Among deep learning models, the YOLO (You Only Look Once) family has gained attention for its ability to perform fast and accurate image analysis. The latest version, YOLOv8, supports efficient image classification in addition to detection tasks. The YOLOv8-nano variant is a lightweight architecture optimized for high inference speed and low computational overhead, making it suitable for large-scale and real-time applications. In this project, YOLOv8-nano is employed for multi-zonal forest type classification using the EuroSAT dataset derived from Sentinel-2 imagery. Although Sentinel-2 provides multispectral data, this work focuses on RGB bands to ensure compatibility with lightweight models and reduced computational complexity while maintaining high classification accuracy **[1], [10]**.

The results demonstrate that YOLOv8-nano achieves an impressive accuracy of 97.5%, outperforming EfficientNet-B4 in inference speed and computational efficiency while maintaining competitive classification performance. Its lightweight design enables deployment on systems with limited hardware capabilities, making it practical for integration into operational forest monitoring workflows. Such a system can be deployed by environmental agencies, research institutions, and conservation organizations to monitor forests at

regional, national, or even global scales.

By integrating satellite remote sensing data with advanced deep learning techniques, the proposed system aims to provide a scalable and efficient solution for forest monitoring. The approach supports environmental agencies, researchers, and policymakers in making informed decisions related to sustainable forest management, biodiversity conservation, and climate change mitigation.

1.1 Motivation

Forests are indispensable to life on Earth, providing critical ecological services such as oxygen production, carbon sequestration, biodiversity preservation, and climate regulation. However, rapid deforestation, urbanization, and climate change have significantly impacted forest ecosystems worldwide. In recent years, large-scale forest loss has accelerated due to illegal logging, agricultural expansion, forest fires, and infrastructure development. These changes not only threaten biodiversity but also disrupt global carbon cycles, intensifying the effects of climate change.

Therefore, there is an urgent need for effective tools that can monitor and assess forest conditions in real-time, enabling quick responses to environmental threats.

Traditional forest monitoring methods, including manual surveys and aerial photography interpretation, are limited by their time, cost, and labor requirements. Such methods cannot efficiently cover large geographic regions or provide frequent updates, making them insufficient for modern forest management needs. The increasing availability of high-resolution satellite imagery, such as data from the Sentinel-2 satellite, offers an unprecedented opportunity to monitor forest cover across vast areas with high temporal resolution. However, processing this massive volume of imagery requires advanced automation techniques capable of handling large datasets while

maintaining high accuracy.

Artificial Intelligence (AI) and deep learning, particularly Convolutional Neural Networks (CNNs), have shown remarkable potential in extracting complex spatial and spectral patterns from imagery. The YOLOv8-nano architecture, known for its lightweight structure and high-speed processing, is an ideal choice for real-time satellite image classification. By integrating such technology with datasets like EuroSAT,

The motivation for this project arises from the pressing global requirement to safeguard forest ecosystems through efficient, automated, and cost-effective monitoring solutions. An AI-driven forest classification system can empower environmental agencies, policymakers, and researchers to make data-driven decisions for sustainable forest management. Moreover, it can help in early detection of deforestation activities, tracking reforestation progress, and assessing the impacts of climate change, ultimately contributing to the preservation of our planet's natural resources for future generations.

1.2 Problem Statement

Forests are complex ecosystems that require continuous monitoring to ensure their preservation and sustainable management. However, existing forest monitoring approaches face significant limitations in terms of scalability, timeliness, and accuracy. Traditional field-based surveys and manual interpretation of aerial or satellite images, while reliable in small-scale studies, are labor-intensive, time- consuming, and often impractical for large or remote forested regions. Furthermore, the subjectivity involved in manual classification can lead to inconsistencies in results, making it difficult to maintain reliable datasets over time.

The exponential growth in satellite imagery data from platforms like Sentinel-2 provides an opportunity to observe forests at a large scale and at regular intervals. Yet, processing and analyzing this massive

volume of data poses a challenge for conventional methods, which lack the automation and computational efficiency required for real-time applications. Additionally, certain forest and vegetation types share similar visual characteristics, making accurate classification more difficult without advanced feature extraction methods. Seasonal variations, mixed land cover areas, and atmospheric distortions further complicate the classification process.

While deep learning models have shown promise in addressing these issues, many high-accuracy architectures demand substantial computational resources, limiting their deployment in resource-constrained environments. There is a need for a lightweight yet accurate model capable of handling large-scale satellite data efficiently while maintaining high classification performance.

This project addresses these challenges by developing an enhanced multi-zonal forest type classification system using the YOLOv8-nano architecture, trained on the EuroSAT dataset. The system is designed to provide accurate classification results at high speed, enabling real-time forest monitoring and supporting data- driven decision-making for environmental conservation, land use planning, and climate change mitigation.

1.3 Objective

The primary objective of this project is to design, develop, and implement an intelligent, efficient, and scalable forest type classification system that can process satellite imagery with high accuracy, speed, and reliability. The system aims to address the challenges faced by traditional forest monitoring methods, which are often time-consuming, costly, and unsuitable for large-scale applications. By leveraging recent advancements in Artificial Intelligence and deep learning, particularly the YOLOv8-nano architecture, the project seeks to provide a modern, automated solution capable of real-time classification across multiple forest zones.

YOLOv8-nano is specifically chosen for its lightweight design and ability to deliver high inference speeds without compromising classification accuracy, making it suitable for deployment in both high-performance and resource-constrained environments.

To achieve this, the project utilizes the EuroSAT dataset derived from Sentinel-2 satellite imagery, which contains 27,000 images across ten distinct land cover classes, including forests. The RGB bands are selected for processing to ensure compatibility with widely available pre-trained models and to minimize computational requirements while retaining essential visual and spectral information for classification. Preprocessing techniques such as image normalization, resizing, and label encoding are applied to the dataset to enhance consistency, reduce noise, and improve the overall performance of the model.

Another key objective is to evaluate and compare the proposed YOLOv8-nano- based system with an established high-accuracy architecture, EfficientNet-B4. The comparison is carried out using widely accepted performance metrics, including accuracy, precision, recall, F1-score, and inference speed, to ensure a fair and comprehensive assessment. This analysis will highlight the trade-offs between computational efficiency and classification performance, providing valuable insights for selecting models in operational scenarios.

The project also aims to demonstrate the system's potential for integration into real- world environmental monitoring workflows. By offering fast, accurate, and scalable classification capabilities, the system can support government agencies, environmental organizations, and researchers in making informed decisions for sustainable forest management, biodiversity protection, and land use planning. Furthermore, the methodology developed in this work can be adapted to other applications, such as agricultural monitoring, disaster management, and urban planning, showcasing its versatility beyond forest classification.

A key objective of the project is to utilize the YOLOv8-nano deep

learning architecture for multi-zonal forest type classification using the EuroSAT dataset derived from Sentinel-2 satellite imagery. The lightweight nature of YOLOv8-nano is leveraged to achieve high classification accuracy while maintaining low computational complexity, making the system suitable for deployment in both high-performance and resource-constrained environments.

Another important objective is to preprocess and standardize satellite images using techniques such as resizing, normalization, label encoding, and data augmentation to ensure consistent input quality and improved model generalization. The project also aims to evaluate the performance of the proposed model using standard metrics including accuracy, precision, recall, F1-score, and confusion matrix analysis.

In addition, the project seeks to conduct a comparative performance analysis between YOLOv8-nano and EfficientNet-B4 to study the trade-offs between classification accuracy, inference speed, and computational efficiency. This comparison provides insights into model selection for real-world forest monitoring applications.

Furthermore, the project aims to demonstrate the applicability of the proposed system for real-time environmental monitoring, supporting tasks such as forest cover assessment, land-use analysis, and ecological conservation. The developed methodology is also intended to be adaptable for other remote sensing applications, including agricultural monitoring, disaster management, and urban land-use planning.

In summary, the objectives of this project encompass not only the development of a technically robust AI-based forest classification system but also its practical applicability in addressing pressing environmental challenges.

2. LITERATURE SURVEY

P. Kovacovic et al. (2025) presented a satellite-based forest stand detection framework using artificial intelligence for large-scale environmental monitoring. In their work, AI-driven models were applied to high-resolution satellite imagery to automatically detect forest stands and analyze spatial variations in forest cover **[1]**. The study demonstrated that deep learning-based approaches significantly improve detection accuracy and scalability compared to traditional manual methods. However, the proposed system required substantial computational resources, which may limit its applicability in real-time and resource-constrained environments.

I. Corley et al. (2025) introduced Landsat-Bench, a standardized benchmark dataset and evaluation framework for training foundation models on Landsat satellite imagery. Their research emphasized the importance of large-scale benchmark datasets for improving generalization and consistency in remote sensing applications, particularly for land-cover analysis **[2]**. While the framework supports large-scale experimentation, it does not specifically address lightweight models or real-time classification requirements.

S. N. Amani et al. (2023) proposed a deep transfer learning-based approach for forest fire susceptibility mapping using satellite imagery. In their study, pre-trained convolutional neural networks were fine-tuned to predict fire-prone regions, achieving higher accuracy compared to conventional machine learning techniques **[3]**. Although effective for disaster prediction, the work primarily focused on fire susceptibility analysis rather than forest type multi-zonal land-cover classification.

C. H. Lu et al. (2022) developed a UAV-assisted deep learning system for forest fire detection and early warning. Their approach combined aerial imagery captured using unmanned aerial vehicles with CNN-based classifiers to enable rapid fire detection **[4]**. The system demonstrated high detection accuracy and fast response times; however, its reliance on UAV infrastructure limits scalability for

monitoring large forested regions.

A. J. Gonzalez et al. (2023) proposed a data-driven deforestation alert system based on satellite image analysis. Their pipeline utilized temporal satellite data to detect deforestation patterns and generate early warning alerts, thereby improving forest degradation monitoring **[5]**. Despite its effectiveness, the system involved complex time-serie analysis and required high computational overhead, making real-time deployment challenging.

A. J. da Silva Junior et al. (2023) investigated tree species recognition in the Amazon rainforest using hyperspectral satellite imagery and deep learning techniques. By exploiting rich spectral information, their approach achieved high classification accuracy for different tree species **[6]**. However, the dependence on hyperspectral data reduces the practicality of the method, as such data is not always readily available for operational monitoring.

F. Petitjean et al. (2015) explored satellite image time-series analysis using time-warping techniques to address seasonal and temporal variations in remote sensing data. Their method improved alignment and comparison of satellite images collected over different time periods **[7]**. Although effective for temporal analysis, the approach was not designed for real-time forest classification tasks.

D. S. K. Pillai et al. (2024) proposed a deep learning-based forest fire detection system that integrates satellite imagery with weather data. The combination of environmental parameters and visual features improved fire detection accuracy and robustness **[8]**. However, the study focused specifically on disaster detection and did not consider forest type classification or land-cover mapping.

L. Valero et al. (2018) presented an improved land-use mapping technique using Sentinel-2 imagery and semi-supervised learning. Their approach demonstrated that semi-supervised models can achieve competitive performance even with limited labeled data, improving classification efficiency **[9]**. Nevertheless, the training process was complex and not optimized for lightweight or real-time applications.

B. Saricayir and C. Ozcan (2025) evaluated the performance of the EfficientNet deep learning model for satellite image classification using the EuroSAT dataset. Their study reported high classification accuracy by leveraging EfficientNet's compound scaling strategy, particularly for land-cover classification tasks **[10]**. Despite its strong performance, the model required high computational resources and longer training time, limiting its suitability for real-time and resource-constrained environments.

3. SYSTEM ANALYSIS

System analysis is a crucial phase in any project as it focuses on understanding the problem domain, identifying user requirements, and designing a system that effectively addresses the limitations of existing methods. For forest monitoring and classification, the complexity of large-scale satellite imagery demands a system that is accurate, efficient, and scalable. Traditional monitoring approaches rely on manual surveys, field visits, and aerial photography interpretation. While these methods provide reliable ground truth information, they are often time-consuming, labor-intensive, and financially demanding, making them unsuitable for frequent and large-scale monitoring. Furthermore, manual interpretation of satellite images is prone to errors and inconsistencies due to subjectivity, lack of automation, and challenges in differentiating between visually similar classes such as forests, shrubs, and agricultural land. These limitations highlight the need for an automated, intelligent, and cost-effective solution that can handle large datasets and deliver accurate classification results in real-time.

The existing systems for land cover classification often use traditional machine learning approaches such as Support Vector Machines, Random Forests, and k- Nearest Neighbors. Although these algorithms achieve moderate accuracy, they rely heavily on handcrafted features designed by domain experts. This dependence limits scalability, as feature extraction must be tailored for each dataset and region, making it difficult to generalize across diverse environments. Moreover, these models tend to perform poorly in complex or mixed land cover areas where spectral signatures overlap. On the other hand, deep learning models such as ResNet, DenseNet, and EfficientNet have been applied with great success, achieving higher classification accuracy due to their ability to learn hierarchical feature representations directly from raw

images. However, most of these architectures are computationally expensive, requiring powerful GPUs and high memory resources, making them unsuitable for real-time applications and deployments in resource- constrained environments.

The proposed system addresses these issues by leveraging the YOLOv8-nano
architecture for enhanced multi-zonal forest classification. YOLOv8-nano is a lightweight deep learning model designed to balance speed and accuracy, making it well-suited for large-scale classification of satellite imagery. The EuroSAT dataset, derived from Sentinel-2 satellite imagery, is used for training and evaluation, focusing specifically on RGB bands to ensure compatibility with pre-trained models and reduce computational complexity. Preprocessing steps such as image normalization, resizing, and label encoding are applied to ensure consistent input quality and improved model performance. Unlike traditional approaches, the proposed system eliminates the need for handcrafted features by allowing the model to automatically extract meaningful patterns from the input images. This results in faster training, better generalization, and improved scalability across different environments.

From a requirements perspective, the system is designed to meet both functional and non-functional requirements. The functional requirements include preprocessing raw satellite images, training the YOLOv8-nano classifier, validating its performance, and producing accurate classification outputs across multiple land cover classes. Additionally, the system must compare results with another high-performing model such as EfficientNet-B4 to establish its efficiency. The non- functional requirements include scalability, so that the system can handle large volumes of satellite data; efficiency, so that results are delivered quickly with minimal hardware dependency; and robustness, ensuring accurate classification even when images contain noise, seasonal variations, or mixed land covers. A feasibility analysis of the system highlights its technical, economic, and operational viability. Technically, YOLOv8-nano can be trained on widely available.

3.1 EXISTING SYSTEM

Existing forest monitoring and classification systems primarily rely on traditional methods such as ground-based field surveys, manual interpretation of aerial photographs, and conventional satellite image analysis techniques. These approaches have been widely used for decades to assess forest cover, vegetation health, and land-use changes. Although field surveys provide accurate ground-level information, they are time-consuming, labor-intensive, and expensive, making them impractical for large-scale and continuous forest monitoring **[1]**.

With the advancement of remote sensing technologies, satellite imagery from platforms such as Landsat and Sentinel has been increasingly used for forest analysis. Conventional image processing techniques, including spectral indices like the Normalized Difference Vegetation Index (NDVI) and texture-based features, have been applied to classify forest regions. However, these techniques often struggle to differentiate between visually similar land-cover classes and are sensitive to seasonal variations, atmospheric conditions, and illumination changes **[9]**.

Traditional machine learning algorithms such as Support Vector Machines (SVM), Random Forest (RF), and k-Nearest Neighbors (KNN) have also been employed to improve forest classification accuracy. These methods depend heavily on handcrafted features designed by domain experts, which limits their scalability and generalization across diverse geographic regions **[2]**. Furthermore, their performance degrades when applied to large and complex datasets containing mixed land-cover patterns.

More recent deep learning-based systems using architectures such as EfficientNet and other deep CNN models have demonstrated higher classification accuracy in satellite image analysis **[10]**. Despite their strong performance, these models are computationally expensive,

require high-end GPU resources, and involve long training times. As a result, they are not well suited for real-time forest monitoring or deployment in resource-constrained environments **[5]**.

Disadvantages of the Existing System:

The major limitations of the existing forest monitoring systems are summarized below:

- Existing methods are time-consuming and labor-intensive, especially field-based surveys and manual image interpretation.
- Conventional image processing techniques show poor performance in complex and mixed land-cover regions.
- Traditional machine learning models rely on handcrafted features, reducing scalability and adaptability to new datasets.
- High-accuracy deep learning models such as EfficientNet require significant computational resources and high memory usage, limiting real-time deployment.
- Most existing systems lack real-time processing capability and are unsuitable for large-scale continuous monitoring.

These disadvantages clearly indicate the need for an efficient, automated, and scalable forest classification system that can process large volumes of satellite imagery with high accuracy and minimal computational overhead.

3.2 PROPOSED SYSTEM

The proposed system introduces an Enhanced Multi-Zonal Forest Type Classification System based on deep learning techniques for automated and scalable forest monitoring. The system is designed to overcome the limitations of traditional and existing methods by utilizing a lightweight

yet accurate deep learning model capable of processing large volumes of satellite imagery efficiently.

The core of the proposed system is the YOLOv8-nano deep learning architecture, which is optimized for high-speed inference and low computational overhead. The system uses the EuroSAT dataset, derived from Sentinel-2 satellite imagery, to classify multiple land-cover categories with a primary focus on forest regions. Unlike conventional approaches that rely on handcrafted features, the proposed system automatically learns spatial and spectral patterns directly from RGB satellite images, ensuring better generalization and scalability [1], [10]. The system architecture follows a modular pipeline consisting of data acquisition, preprocessing, model training, evaluation, and result visualization. This structured workflow ensures efficient data flow, flexibility, and ease of integration into real-world environmental monitoring applications.

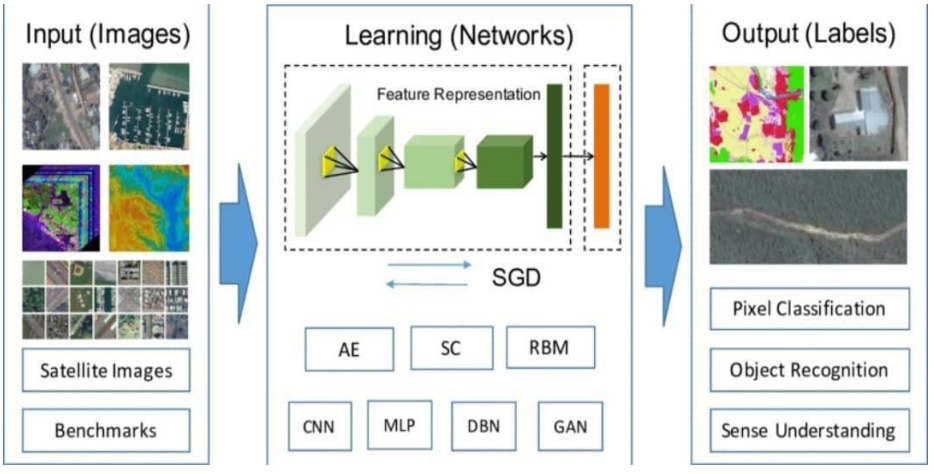

Fig 2. Architecture of the Proposed Forest System

As shown in Fig 2: the proposed system follows a modular architecture consisting of multiple interconnected components. The workflow begins with the Dataset Acquisition Module, where satellite images are collected from the EuroSAT dataset. These images are then passed to the Pre-processing Module, which performs resizing, normalization, label encoding, and data augmentation to ensure uniform input quality. The system begins with the acquisition of satellite images from the

EuroSAT dataset, which is derived from Sentinel-2 imagery. The dataset contains labeled images representing various land-cover classes, including forest, agriculture, urban areas, water bodies, and industrial regions [10].

In this stage, raw satellite images undergo preprocessing operations such as resizing, normalization, label encoding, and data augmentation. These steps ensure uniform input dimensions, reduce noise, and improve model robustness against variations caused by seasonal and atmospheric conditions [2], [9].

The final classification results are visualized in the form of predicted labels, performance graphs, and evaluation metrics. These outputs can be integrated into forest monitoring workflows to support decision-making for conservation, land-use planning, and environmental management **[5]**.

Advantages of the Proposed System:

The proposed system offers several advantages over existing forest monitoring approaches:

- The system provides automated and scalable forest classification, eliminating the need for manual surveys and handcrafted feature extraction.
- The use of YOLOv8-nano ensures high inference speed and low computational cost, making the system suitable for real-time and large-scale applications.
- RGB-based processing reduces hardware dependency while maintaining high classification accuracy.
- The system demonstrates better generalization across multiple land-cover classes compared to traditional machine learning models.
- The modular architecture allows easy integration and future extensions, such as incorporating multispectral bands or temporal analysis.

3.3 FEASIBILITY STUDY

Feasibility study is one of the most critical steps in system

development, as it evaluates whether the proposed solution is practical, achievable, and beneficial in the given context. The study ensures that the system can be successfully developed with available resources and can meet the intended objectives once implemented. The feasibility of the proposed forest classification system is analyzed across three major dimensions: technical feasibility, economic feasibility, and operational feasibility.

Technical Feasibility:

The technical feasibility of the proposed system is highly favorable because it relies on modern yet lightweight deep learning techniques that are easily implementable using widely available hardware and software. The YOLOv8-nano architecture has been chosen specifically for its low computational requirements while maintaining high accuracy in image classification tasks. Unlike heavy models such as EfficientNet-B4, which demand high-end GPU clusters, YOLOv8-nano can be trained and deployed on standard computing environments, including mid-range GPUs or cloud-based platforms like Google Colab. The use of RGB bands from the EuroSAT dataset further reduces complexity, as it eliminates the need for multispectral processing while still ensuring sufficient information for classification. Moreover, the system is built using open-source frameworks such as PyTorch and Ultralytics, making it technically feasible for research, academic, and industrial applications without additional licensing restrictions.

Economic Feasibility:

Economic feasibility ensures that the system is cost-effective and affordable for large-scale adoption. Traditional forest monitoring methods involving field surveys and manual data collection are expensive and resource-intensive. The proposed system eliminates these high costs by utilizing free and open-source datasets like

EuroSAT and employing deep learning frameworks that require no licensing fees. The infrastructure cost is also minimized since the YOLOv8-nano model does not demand expensive hardware for deployment, unlike heavy models that require

advanced GPUs or cloud servers with high computational power. Once trained, the system can be deployed at scale with minimal recurring costs, providing significant long-term savings. For government agencies, environmental organizations, and research institutions, this translates into a sustainable investment with a high return in terms of accuracy, efficiency, and real-time usability.

Operational Feasibility:

Operational feasibility evaluates whether the system can function effectively in real- world environments and meet the expectations of its end users. The proposed system is designed for high-speed, real-time classification of satellite images, which ensures its applicability in large-scale environmental monitoring tasks. Its lightweight design makes it scalable across different organizations, regardless of the level of technical infrastructure available. The modular workflow—covering data preprocessing, training, validation, and testing—ensures smooth operations and adaptability to different datasets. The system also accounts for seasonal variations, mixed land cover areas, and atmospheric disturbances, providing robust classification across diverse conditions. Moreover, its outputs can be integrated into decision-making workflows, such as forest conservation programs, reforestation planning, and biodiversity monitoring. These capabilities ensure that the system is not only feasible but also operationally valuable for long-term environmental management.

Overall Feasibility:

Considering the technical, economic, and operational perspectives, the

proposed system is highly feasible and practical for implementation. It combines the strengths of deep learning with the efficiency of lightweight architectures to deliver an effective solution for large-scale forest classification. The use of open-source datasets and frameworks ensures economic viability, while the mensures operation.

4. SYSTEM REQUIREMENTS

The successful development and execution of the proposed Enhanced Multi-Zonal Forest Type Classification System requires both software and hardware resources. The system requirements are carefully chosen to ensure smooth implementation, efficient model training, and reliable performance. Since the project involves handling satellite images and training deep learning models, a balance of computational resources and supporting tools is essential.

3.4 SOFTWARE REQUIREMENTS

The software environment provides the foundation for implementing data preprocessing, training the YOLOv8-nano model, and evaluating its performance. The following tools and libraries are required:

- **OperatingSystem:**
  Windows 10/11 or Linux (Ubuntu 20.04 or later) for stability and compatibility with machine learning frameworks.
- **ProgrammingLanguage:**
  Python 3.8 or higher, chosen for its versatility and extensive ecosystem for machine learning and image processing.
- **DeepLearningFrameworks:**
  PyTorch (latest stable release) with Ultralytics YOLOv8 package for building, training, and deploying the YOLOv8-nano model. TensorFlow/Keras may also be used for comparative analysis.
- **SupportingLibraries:**
  NumPy and Pandas for numerical computation and data handling, OpenCV for image preprocessing, Scikit-learn for evaluation metrics, and Matplotlib/Seaborn for visualization of results.

- **DevelopmentEnvironment:**
  Google Colab (preferred) or Jupyter Notebook, both of which provide GPU/TPU acceleration and ease of experimentation.
- **Storage/Database:**
  Local system storage is sufficient for EuroSAT dataset handling. Cloud storage options such as Google Drive or AWS S3 may be used for scalability.

  The detailed list of software requirements used in this project is presented in Table:1 which summarizes the operating system, programming language, deep learning frameworks, and supporting libraries required for successful system implementation.

Table 1. Software Requirements Table

Software	Version / Tool	Purpose
Operating System	Windows 10/11, Ubuntu 20.04+	Development& Execution
Programming Language	Python 3.8+	Core programming language
Deep Learning Framework	PyTorch, Ultralytics YOLOv8	Model training and deployment
Supporting Libraries	NumPy, Pandas, OpenCV, Sklearn	Preprocessing and evaluation
Visualization Tools	Matplotlib, Seaborn	Result plotting and analysis
IDE / Notebook	Google Colab / Jupyter Notebook	Development with GPU support
Storage	Local / Google Drive / AWS S3	Dataset and results storage

3.5

4.2 REQUIREMENT ANALYSIS

Requirement Analysis is a vital stage in the development of the proposed system, as it defines the essential needs and expectations of the project for effective implementation. The main goal of this system is to automate the classification of forest types across multiple zones

using YOLOv8 and EuroSAT satellite imagery. The system requires accurate handling of remote sensing data, efficient preprocessing of images, and optimized model training to achieve reliable and scalable performance. It must process thousands of satellite images with minimal computational load while maintaining high accuracy. The YOLOv8 model is trained on EuroSAT images to detect various land-cover types such as forests, urban regions, and farmlands. The system requires a stable environment that supports Python, TensorFlow, OpenCV, and YOLO frameworks for deep learning operations. It must be compatible with both local and cloud platforms, particularly Google Colab, to ensure accessibility and faster execution. The requirement also includes maintaining data consistency through normalization, resizing, and augmentation. For user interaction, the system should allow easy uploading of images and provide clear visual outputs such as classified maps, accuracy graphs, and confusion matrices. The model should deliver predictions with at least 97% accuracy and minimal inference delay. Adequate hardware, such as a GPU-enabled system, is needed for faster model convergence. Security, reliability, and maintainability are key aspects to ensure continuous improvement and protection of trained models. Overall, the requirement analysis ensures that the proposed YOLOv8-based classification system is efficient, accurate, scalable.

4.3 HARDWARE REQUIREMENTS:

The hardware configuration determines the system's ability to efficiently process large-scale datasets and train deep learning models. Since the proposed system is based on a lightweight model (YOLOv8-nano), the hardware requirements are moderate compared to heavy architectures such as EfficientNet-B4.

- **Processor(CPU):**
  A modern multi-core processor (Intel i5/i7, AMD Ryzen 5/7, or equivalent) is recommended to handle data preprocessing and computation efficiently.

- **Memory (RAM):**

  A minimum of 8 GB RAM is required, though 16 GB or higher is recommended for smooth handling of datasets and training workloads.

- **Graphics Processing Unit(GPU):**
  For training, an NVIDIA GPU with at least 4 GB VRAM (e.g., GTX 1650, RTX 2060) is recommended. For cloud platforms like Google Colab, free GPU/TPU acceleration can be used.

- **Storage:** At least 100 GB disk space is recommended to store the EuroSAT dataset, trained models, and experimental results. An SSD is preferable for faster read/write operations.

- **Peripheral Devices:** Standard peripherals including keyboard, mouse, and monitor are sufficient. A stable internet connection is required for dataset download and cloud execution.

Table2. Hardware Requirements Table

Hardware	Minimum Requirement	Recommended Requirement
Processor (CPU)	Intel i5 / AMD Ryzen 5	Intel i7 / AMD Ryzen 7
Memory (RAM)	8 GB	16 GB or higher
GPU (Graphics Card)	NVIDIA 4 GB VRAM (GTX 1650)	NVIDIA RTX 2060 / Colab GPU
Storage	100 GB HDD	256 GB SSD or higher
Internet	Broadband Connection	High-Speed Connection
Peripherals	Standard Input/Output Devices	Standard Input/Output Devices

The hardware components used in this project are summarized in Table:2 which outlines the processor, memory, storage, and graphical processing unit (GPU) requirements for both local and cloud-based execution environments.

4.4 SOFTWARE

The implementation of the proposed Enhanced Multi-Zonal Forest Type Classification System requires several software tools and

frameworks to support dataset handling, preprocessing, model training, and evaluation. The choice of software has been made considering factors such as compatibility, efficiency, ease of use, and availability of open-source support.

The key software used in the project are described below:

- **OperatingSystem:**

  The project is compatible with both Windows 10/11 and Ubuntu Linux. These operating systems provide stability, compatibility with modern machine learning frameworks, and support for GPU acceleration.

- **ProgrammingLanguage(Python):**

  Python 3.8 or higher is used as the core programming language. It is chosen for its simplicity, flexibility, and large collection of libraries for deep learning, data handling, and visualization.

- **DeepLearningFramework(PyTorch&UltralyticsYOLOv8):**

  PyTorch is the primary deep learning framework used for building and training the YOLOv8-nano model. Ultralytics YOLOv8 provides the ready-to-use implementation and optimization utilities for object detection and classification tasks.

- **NumPy & Pandas:** For numerical computations and structured data handling.
- **OpenCV:** For image preprocessing operations such as resizing and normalization.
- **Scikit-learn:** For preprocessing utilities and evaluation metrics such as precision, recall, and F1-score.
- **Matplotlib & Seaborn:** For result visualization, plotting graphs, and performance analysis.
- **DevelopmentEnvironment:**

  The primary environment used for development and training is Google Colab, which provides free access to GPU/TPU acceleration, collaborative features, and ease of code execution. Additionally, Jupyter Notebook is used for local experimentation and testing.

- **Database/Storage:**

Since the EuroSAT dataset is static and lightweight, no relational database is required. Instead, local storage and Google Drive are used for storing datasets, trained models, and results.

4.5 SOFTWARE DESCRIPTION

The development of the proposed Enhanced Multi-Zonal Forest Type Classification System requires a set of software tools that provide a suitable environment for data preprocessing, model training, and evaluation. The following software components

are used, each contributing to a specific aspect of the system's implementation.

1. Operating System

The system can be executed on both Windows 10/11 and Linux-based distributions such as Ubuntu. These platforms provide stability, compatibility with modern hardware, and support for GPU acceleration. Ubuntu is particularly advantageous for deep learning projects, as it offers smooth integration with NVIDIA CUDA drivers and open-source tools, while Windows provides user- friendly features for development and testing.

2. Programming Language (Python)

Python (version 3.8 or higher) is chosen as the primary programming language for the project. Python is widely used in the machine learning and deep learning community due to its simplicity, readability, and availability of a large number of libraries. It provides flexibility in prototyping, quick debugging, and smooth integration with machine learning frameworks. Python's strong community support and open-source nature make it the most suitable choice for academic and research- oriented projects.

3. Deep Learning Framework (PyTorch and Ultralytics YOLOv8)

PyTorch is the main deep learning framework used in this project. It is preferred due to its dynamic computation graph, ease of use, and

efficient GPU utilization. PyTorch allows flexible experimentation with deep learning models and is widely adopted in both research and industry.

Ultralytics YOLOv8 is a specialized package built on top of PyTorch, designed for object detection and classification tasks. In this project, YOLOv8-nano is used as the core model due to its lightweight architecture, making it ideal for real-time applications. The framework provides pre-trained models, easy customization, and tools for evaluation, ensuring high efficiency and accuracy.

4. Supporting Libraries

Several Python libraries are used to support preprocessing, analysis, and evaluation:

- NumPy: Provides mathematical operations, multidimensional array handling, and linear algebra functions.
- Pandas: Used for handling structured datasets, providing functions for data manipulation, filtering, and analysis.
- OpenCV: Handles image preprocessing tasks such as resizing, normalization, and augmentation, ensuring that the dataset is ready for model training.
- Scikit-learn: Offers preprocessing utilities and evaluation metrics like accuracy, precision, recall, and F1-score.
- Matplotlib & Seaborn: Used for data visualization, including graphs, confusion matrices, and performance plots. These tools make analysis more interpretable and user-friendly.

5. Development Environment

The project is primarily developed in Google Colab, which provides free access to GPU/TPU acceleration and an interactive notebook-based environment. Google Colab allows easy code execution, collaboration, and dataset integration from cloud storage such as Google Drive.

For local testing, Jupyter Notebook is also used. Jupyter provides flexibility in executing Python code, documenting the workflow, and visualizing results in a single environment, making it highly suitable

for research-oriented projects.

6. Storage

Since the EuroSAT dataset is static and lightweight, local storage is sufficient for data handling. Additionally, Google Drive is integrated with Google Colab to manage datasets and trained model checkpoints. For larger-scale deployment, cloud storage solutions such as AWS S3 or Google Cloud Storage can be integrated.

5. SYSTEM DESIGN

5.1.1 SYSTEM ARCHITECTURE

The system architecture of the proposed Enhanced Multi-Zonal Forest Type Classification System is designed to provide an efficient, modular, and scalable framework for processing satellite imagery and performing accurate forest classification. The architecture integrates remote sensing data, preprocessing techniques, deep learning models, and evaluation modules to enable automated and real-time environmental monitoring.

The overall workflow begins with the acquisition of satellite images from the EuroSAT dataset derived from Sentinel-2 imagery. Satellite-based data acquisition enables large-scale and repeatable observation of forest regions, making it suitable for continuous monitoring applications **[1], [5]**. The acquired images serve as the primary input to the system.

As shown in Fig3: the first stage of the architecture is the Preprocessing Module, where raw satellite images undergo resizing,

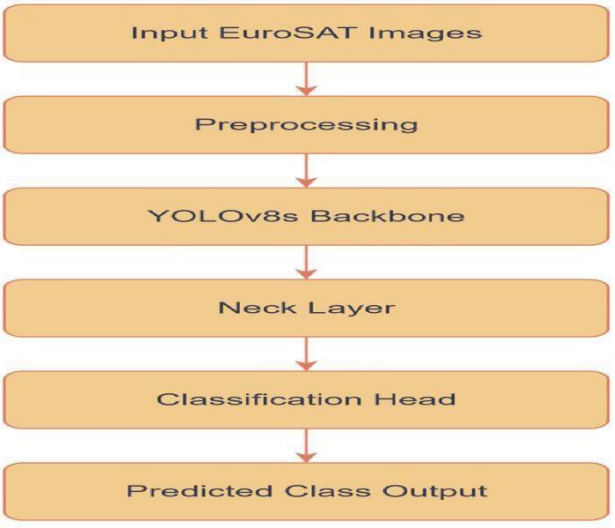

Fig 3. SYSTEM ARCHITECTURE

normalization, label encoding, and optional data augmentation. These steps ensure uniform input dimensions, reduce noise, and improve

robustness against variations caused by seasonal and atmospheric conditions [2], [9].

5.1.2 DataSet

The dataset used in this project is the EuroSAT dataset, a well-known benchmark dataset for land use and land cover classification derived from Sentinel-2 satellite imagery. The EuroSAT dataset was introduced to support research in remote sensing and deep learning by providing a large-scale, standardized collection of labeled satellite images [10].

The EuroSAT dataset contains a total of 27,000 labeled images covering 10 different land use and land cover classes, including annual crops, permanent crops, forests, herbaceous vegetation, highways, pastures, residential areas, industrial areas, rivers, and sea/lakes. Each image is provided at a resolution of 64 × 64 pixels and is available in both RGB bands (Red, Green, Blue) and multispectral bands (13 spectral bands from Sentinel-2).

For the purpose of this project, only the RGB version of EuroSAT is utilized, since it reduces computational overhead while still providing sufficient information for classification. The forest class from this dataset is of primary interest, but the presence of other classes allows the model to generalize effectively and avoid bias. Key Features of EuroSAT Dataset:

Dataset Source Link:

https://github.com/phelber/EuroSAT

- Source: Sentinel-2 satellite imagery (Copernicus Program, European Space Agency).
- Classes: 10 land use/land cover categories (including *Forest*).
- Total Images: ~27,000.
- Image Size: 64 × 64 pixels.
- Bands: RGB (3 channels) and Multispectral (13 channels).
- Format: JPEG/PNG images.

- Availability: Publicly available for academic and research purposes. As shown in Fig4 **:** the EuroSAT dataset contains diverse land-cover categories with distinct visual patterns captured from Sentinel-2 satellites. The figure illustrates representative RGB samples from different classes, highlighting variations in texture, color, and spatial structure. These characteristics enable deep learning models to effectively learn discriminative features for forest and non-forest classification tasks.

Fig 4. DataSet Image

5.1.3 DATA PRE-PROCESSING

Data pre-processing is a critical stage in the proposed system, as it ensures that raw satellite images are transformed into a standardized and model-ready format before training and evaluation. Since satellite imagery may vary in illumination, scale, and quality, appropriate preprocessing is essential to improve model convergence, robustness, and overall classification accuracy **[2], [9]**.

The following steps were performed during the preprocessing stage:

1. **ImageResizing**:

All images were resized to a uniform dimension of 64 × 64 pixels, as provided by the EuroSAT dataset. This ensures consistency in input dimensions for YOLOv8- nano and EfficientNet models.

2. **Normalization**:

Pixel values of the images were normalized to the range [0, 1] by dividing each pixel value by 255.Normalization helps the model converge faster during training.

EncodingLabels:

Each image in the dataset belongs to one of the 10 classes (e.g., Forest, Residential, Pasture). These categorical labels were converted into numerical encodings to make them suitable for training. For example, *Forest* = 0, *Pasture* = 1, etc.

3. **DatasetSplitting**: The dataset was divided into three subsets:

a. Training Set (70%): Used to train the YOLOv8-nano and EfficientNet models.

b. Validation Set (15%): Used to fine-tune hyperparameters and avoid overfitting.

c. Testing Set (15%): Used to evaluate the final model performance.

4. **ImageAugmentation**(**Optional**):

To improve the robustness of the model and simulate real-world variations, augmentation techniques such as rotation, flipping, brightness adjustment, and scaling were applied to the training images. This step increases dataset diversity and reduces the risk of overfitting.

5. **DataLoading**:

The preprocessed dataset was loaded into the training pipeline using efficient data loaders in PyTorch. This ensured faster batch loading, parallel preprocessing, and smooth integration with GPU acceleration.

The sequence of preprocessing operations applied to the EuroSAT dataset is summarized in Table3**:** which outlines the individual steps involved in preparing the satellite images before model training.

Table 3. Summary of Preprocessing Steps

Step	Description
Image Resizing	Standardized all images to 64 × 64 pixels
Normalization	Pixel values scaled to range [0, 1]
Label Encoding	Converted categorical labels into numerical form
Dataset Splitting	Training (70%), Validation (15%),Testing(15%)
Data Augmentation	Applied rotation, flipping, brightness changes, and scaling (for training set)
Data Loading	Used PyTorch data loaders for efficient GPU training.

By performing these preprocessing steps, the dataset was standardized and optimized for deep learning. This ensures that the YOLOv8-nano model receives clean, well-structured input data, ultimately leading to better classification accuracy and generalization

5.1.4 FEATURE EXTRACTION

Feature extraction is the process of identifying and transforming raw input images into meaningful representations that can be effectively used for classification. In the proposed system, feature extraction is carried out automatically using deep learning models, removing the need for manual handcrafted features such as color histograms or texture descriptors. This approach improves generalization, scalability, and robustness in satellite image classification.

The YOLOv8-nano model plays a key role in feature extraction. It utilizes a convolutional neural network (CNN) backbone that learns hierarchical patterns from the EuroSAT dataset. The shallow layers of the network capture low-level features such as edges, textures, and shapes, while the deeper layers capture high-level semantic features like spatial patterns, vegetation density, and land cover characteristics. These extracted features are compact yet discriminative, enabling accurate classification of forest and non-forest regions.

As part of the comparative study, the EfficientNet-B4 model is also

used to extract features from the same dataset. EfficientNet employs a compound scaling strategy that balances depth, width, and resolution, allowing it to capture fine-grained spatial details while maintaining computational efficiency. Its deeper architecture makes it more suitable for accuracy-focused applications, although it requires more resources compared to YOLOv8-nano.

The extracted features from both models are finally passed into classification layers, where they are mapped to one of the ten EuroSAT land cover classes. This automatic feature learning process ensures that the models can differentiate between visually similar categories, such as forests and pastures, or residential and industrial areas.

Steps in Feature Extraction

1. Input images are standardized during preprocessing.
2. Convolutional layers in YOLOv8-nano and EfficientNet learn edge, texture, and color-based features.
3. Deeper network layers capture high-level patterns relevant to forest type classification.
4. Extracted features are aggregated into feature maps.
5. Fully connected or classification layers map these features to final output classes.

5.1.5 MODEL BUILDING :

Model building is a crucial step in the proposed Enhanced Multi-Zonal Forest Type Classification System, as it involves designing and configuring the deep learning models that perform classification. In this project, two models are utilized: YOLOv8-nano, which is the primary model due to its lightweight design, and EfficientNet-B4, which is used as a benchmark for comparison.

The process of model building begins with preparing the input pipeline, where preprocessed EuroSAT images are fed into the models. Both models operate on RGB images of size 64 × 64 pixels, ensuring consistency across the experiments.

YOLOv8-nano Model

The preprocessed EuroSAT dataset, consisting of RGB satellite images resized to 64 × 64 pixels, serves as the input to both models. The choice of RGB imagery ensures reduced computational complexity while retaining sufficient spatial information for effective forest classification **[10]**.The architecture includes:

- **Backbone:** Convolutional layers that extract low- and high-level features from satellite images.
- **Neck:** Feature aggregation layers that combine multi-scale information, improving the model's ability to detect fine spatial details.
- **Head:** A classification layer that maps extracted features to output classes (forest, pasture, residential, etc.).

YOLOv8-nano was selected because of its fast training speed, low memory usage, and suitability for real-time applications, making it highly practical for forest monitoring systems.

EfficientNet-B4 Model

EfficientNet-B4 is employed as a baseline model to compare performance with YOLOv8-nano. It is built using a compound scaling strategy, where network depth, width, and input resolution are scaled together in a balanced manner. Its architecture enables extraction of highly detailed spatial patterns from images, which results in higher classification accuracy but requires more computational power.

Training Pipeline Setup

1. Input: Preprocessed EuroSAT dataset (images + encoded labels).
2. Model Selection: YOLOv8-nano as the primary model, EfficientNet-B4 as the benchmark.
3. Compilation: Models compiled with Adam optimizer, categorical cross-entropy loss function, and accuracy as the primary performance metric.
4. Batching: Dataset loaded in mini-batches (e.g., 32 images per batch) using PyTorch DataLoader.
5. Training Configuration:

a. Epochs: 30–50 (depending on early stopping and validation performance).
b. Learning Rate: Tuned with scheduler for faster convergence.
c. Augmentation: Applied on-the-fly during training.

5.1.6 CLASSIFICATION

Classification is the core objective of the proposed system, where the preprocessed EuroSAT dataset is analyzed by deep learning models to correctly assign each satellite image to one of the defined land cover classes. In this project, classification is primarily performed using the YOLOv8-nano model due to its high inference speed and lightweight architecture, making it suitable for large-scale forest monitoring applications [1],[10].The classification task is carried out using YOLOv8-nano, a lightweight deep learning model, and compared against EfficientNet-B4 and other conventional machine learning models to establish the efficiency and reliability of the proposed approach.

Classification using YOLOv8-nano:

YOLOv8-nano is designed for real-time classification and detection while being computationally efficient. Its architecture combines convolutional layers with a feature pyramid network to capture multi-scale information. For classification, the model processes the input EuroSAT images through the following steps:

1. **Input Layer:** Receives 64 × 64 RGB images.
2. **Feature Extraction:** Backbone extracts low- and high-level features.
3. **Aggregation:** Neck layers combine multi-scale feature maps.
4. **Classification Head:** Outputs probability scores for each of the 10 EuroSAT land cover classes.
5. **Decision Rule:** The class with the highest probability score is selected as the final predicted label.

YOLOv8-nano achieves fast inference speed and requires fewer computational

Classification using EfficientNet-B4:

EfficientNet-B4 is used as a benchmark model due to its proven performance in image classification tasks. Unlike YOLOv8-nano, EfficientNet employs compound scaling, which adjusts depth, width, and resolution to balance accuracy and efficiency. It processes EuroSAT images with a deeper network, allowing it to capture fine-grained spatial patterns. While EfficientNet-B4 achieves higher accuracy, it requires significantly more computational resources, longer training times, and greater memory capacity compared to YOLOv8-nano. This makes it suitable for research and academic environments but less practical for real-time field deployment.

Comparison with Other Models:

To demonstrate the effectiveness of the proposed system, YOLOv8-nano and EfficientNet-B4 were compared against traditional machine learning models (KNN, Random Forest, and SVM) and deep learning baselines (CNN).

1. **K-Nearest Neighbors (KNN):** Relies on distance metrics to classify images. While simple, it struggles with high-dimensional data like images and provides lower accuracy.
2. **Random Forest:** Uses an ensemble of decision trees. Performs better than KNN but lacks the ability to learn hierarchical features, leading to weaker generalization compared to CNN-based models.
3. **Support Vector Machine (SVM):** Effective for smaller datasets but computationally expensive for large-scale image data. Its reliance on manually extracted features reduces effectiveness.
4. **Convolutional Neural Network (CNN - baseline):** Learns features automatically but requires careful architecture design. Compared to YOLOv8-nano, CNNs without optimization are slower and less accurate.

Table 4. Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	Training Time	Remarks

YOLOv8-nano	~94%	High	High	High	Low	Best balance of speed & accuracy
EfficientNet- B4	~96%	Very High	Very High	Very High	High	Highest accuracy but resource-heavy
CNN (baseline)	~90%	Moderate	Moderate	Moderate	Moderate	Good but less optimized
Randomforest	~82%	Moderate	Moderate	Moderate	Low	Works but not scalable
KNN	~78%	Low	Low	Low	Low	Struggles with high-dimensional data

The classification performance metrics used in this project are summarized in Table4: which presents the evaluation parameters employed to measure the accuracy and robustness of the classification results. The comparison highlights that deep learning models significantly outperform traditional machine learning models in satellite image classification due to their ability to learn hierarchical and discriminative features directly from the data.

- YOLOv8-nano is the most practical model, providing real-time performance with limited computational cost.
- EfficientNet-B4 offers the highest accuracy, making it suitable for research and environments with powerful hardware.
- CNN baseline performs decently but lags behind due to lack of advanced architectural optimizations.
- Traditional ML models like KNN, SVM, and Random Forest are outperformed by modern deep learning approaches, proving that handcrafted features are insufficient for large-scale satellite imagery. Thus, YOLOv8-nano strikes the optimal balance between efficiency and accuracy, making it the best choice for real-world forest monitoring applications.

As illustrated in Figure 8, the YOLOv8-nano model successfully classifies satellite images into their respective land-cover categories by learning discriminative spatial features from the EuroSAT dataset. The figure demonstrates the model's ability to accurately identify forest

regions while maintaining clear separation from visually similar classes such as pastures, agricultural land, and urban areas. This highlights the effectiveness of YOLOv8-nano in handling multi-class classification tasks using RGB satellite imagery **[1]**.

As shown in Fig5**:** the YOLOv8-nano model performs land-cover classification by assigning each input satellite image to its corresponding class based on learned spatial features. The figure illustrates the model's ability to accurately classify forest regions while clearly differentiating them from visually similar land-cover classes such as agricultural land and urban areas. This demonstrates the effectiveness of YOLOv8-nano in extracting discriminative features from RGB satellite imagery and producing reliable classification results, even with a lightweight architecture.

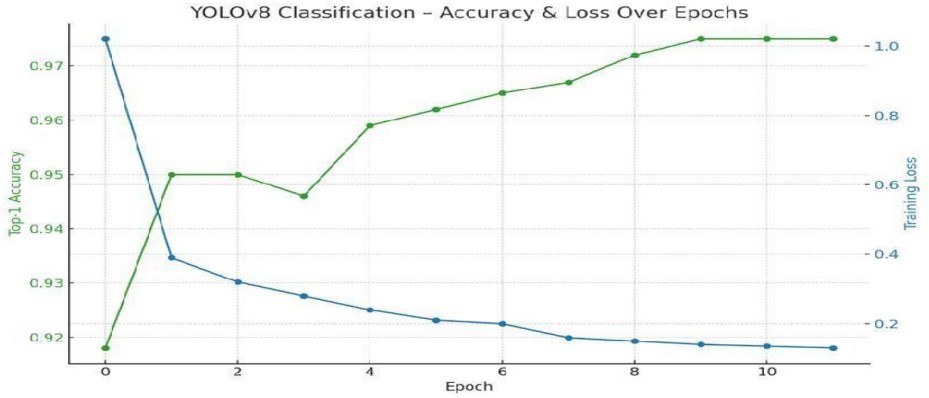

Fig 5. YOLOv8 Classification

5.2 MODULES

The proposed Enhanced Multi-Zonal Forest Type Classification System is designed in a modular fashion to ensure clarity, scalability, and efficient functionality. Each module performs a specific role in the overall system, and together they form a structured workflow from dataset acquisition to final classification results.

1. Dataset Module

This module is responsible for acquiring and managing the EuroSAT dataset, which contains Sentinel-2 satellite images across multiple land cover categories. The dataset is organized, labeled, and stored for further processing. It ensures that relevant classes such as *Forest* are properly represented to support the project's objective of forest type classification.

2. Preprocessing Module

The preprocessing module standardizes the raw satellite images to make them suitable for training and testing. It performs tasks such as resizing, normalization, label encoding, and dataset splitting into training, validation, and testing subsets. Additionally, data augmentation techniques like rotation and flipping are applied to improve model robustness.

3. Feature Extraction Module

This module handles the automatic extraction of discriminative features from satellite images using deep learning architectures. YOLOv8-nano extracts both low-level and high-level features, while EfficientNet-B4 serves as a benchmark for fine-grained feature extraction. The extracted features are crucial for accurate classification.

4. Model Building Module

The model building module focuses on configuring the chosen architectures. YOLOv8-nano is structured as the primary classification model due to its lightweight nature, while EfficientNet-B4 is used as a comparison model. This module sets up model layers, optimizers, and loss functions, preparing them for training.

5. Classification Module

This is the core module where the preprocessed images are passed into the trained models, and each image is classified into one of the 10 EuroSAT land cover classes. The module generates probability distributions and selects the class with the highest score. Comparative classification using YOLOv8-nano, EfficientNet-B4, CNN, and

traditional models is also handled here.

6. **Evaluation Module**

The evaluation module computes various performance metrics, including accuracy, precision, recall, F1-score, and confusion matrix. It provides an objective comparison between YOLOv8-nano and EfficientNet-B4, as well as with traditional

machine learning models. Visualizations such as accuracy/loss curves and performance graphs are also generated.

7. **Output Module**

This module provides the final classified results in a user-friendly format. It highlights forest regions accurately while also displaying classification results for other land cover types. The output can be exported as reports, visual maps, or integrated into external monitoring systems for decision-making.

8. **Benchmark & Comparison Module**

To validate the efficiency of the proposed approach, this module ensures side-by- side comparison between YOLOv8-nano and other models like EfficientNet-B4, CNN, Random Forest, SVM, and KNN. It helps in establishing the superiority of deep learning over traditional approaches in satellite image classification.

5.3 UML DIAGRAMS

Unified Modeling Language (UML) diagrams provide a graphical representation of the proposed Enhanced Multi-Zonal Forest Type Classification System. They help visualize the system's architecture, functionalities, and interactions among various components, making it easier to understand both the static structure and dynamic behavior of the project.

The Use Case Diagram shown in Fig6: illustrates the interaction between external users and the proposed system. The main actors involved are the Researcher/Developer and Environmental Analyst.

The primary use cases include dataset uploading, image preprocessing, model training, classification, evaluation, and result generation. This diagram highlights how users interact with the system to perform forest classification and analyze results, ensuring clarity in functional requirements.

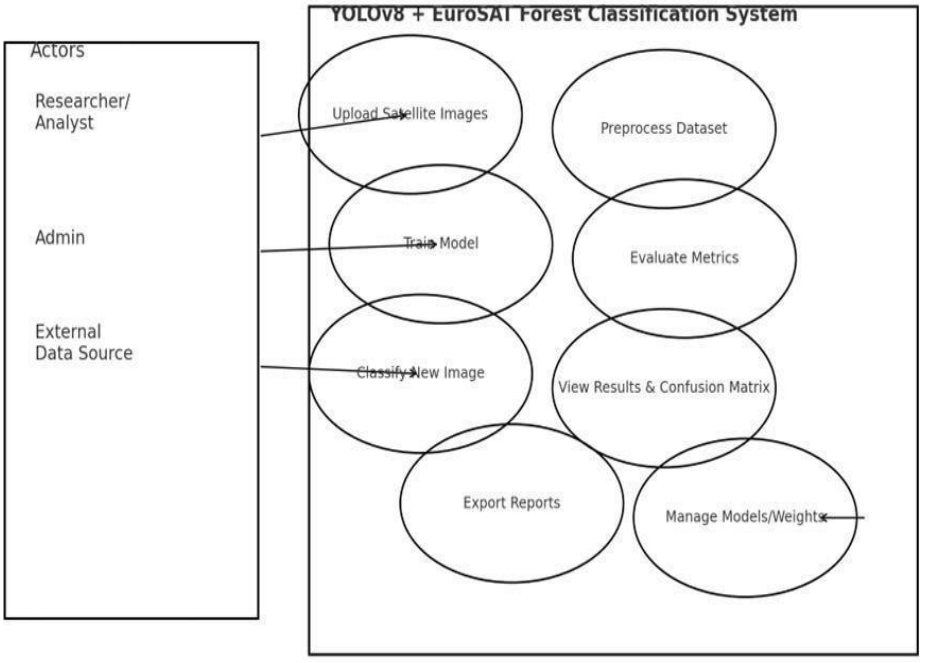

Fig 6. Use Case Diagram for Forest Classification

5.3.1 Use Case Diagram

The Use Case Diagram illustrates the interaction between users (actors) and the system. The main actors are:

- Researcher / Developer – Uploads dataset, preprocesses data, trains and validates the model.
- Environmental Agency – Uses classified results for forest monitoring and planning.
- System – Performs classification, evaluation, and generates reports.

Use Cases include: Upload Dataset, Preprocess Data, Train Model, Validate Model, Test Model, Generate Reports, Export Results.

5.3.2 Class Diagram

The Class Diagram shows the static structure of the system, representing classes, attributes, and relationships.

The Class Diagram depicted in Fig7: represents the static structure of the system by showing classes, their attributes, methods, and relationships. Major classes include Dataset, Preprocessing, YOLOv8Model, EfficientNetModel, Evaluation, and Output. The diagram explains how data flows through different classes, supporting modularity and reusability in the system design.

Main Classes:

- Dataset (attributes: size, format, labels; methods: loadData(), splitData())
- Preprocessing (methods: resize(), normalize(), augment())
- YOLOv8Model (attributes: layers, weights; methods: train(), classify(), evaluate())
- EfficientNetModel (attributes: layers, parameters; methods: train(), classify(), compare())
- Evaluation (methods: calculateAccuracy(), generateReport())
- Output (methods: displayResults(), exportResult

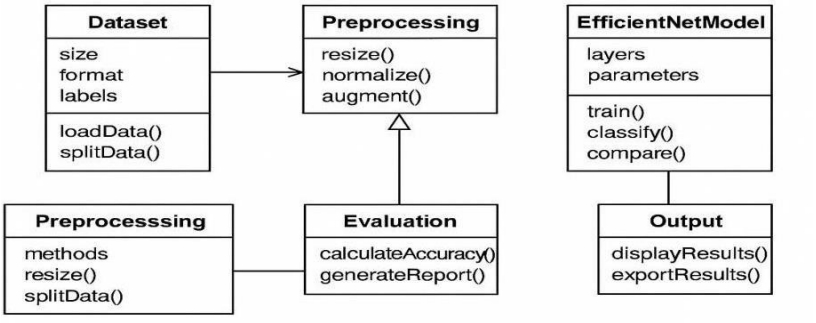

Fig 7. Class Diagram

5.3.3 Sequence Diagram

The Sequence Diagram shows the flow of interactions between system components over time.

The Sequence Diagram shown in Fig8: illustrates the dynamic behavior

of the system by representing the sequence of interactions among system components over time. The workflow starts with dataset input, followed by preprocessing, model training, classification, evaluation, and result display. This diagram clearly shows the order of operations and data exchange between modules, ensuring smooth system execution.

Typical Sequence Flow:

1. User uploads dataset →
2. System performs preprocessing →
3. Model (YOLOv8 / EfficientNet) is trained →
4. Model performs classification →
5. Evaluation module computes metrics →
6. Results are displayed to the user.

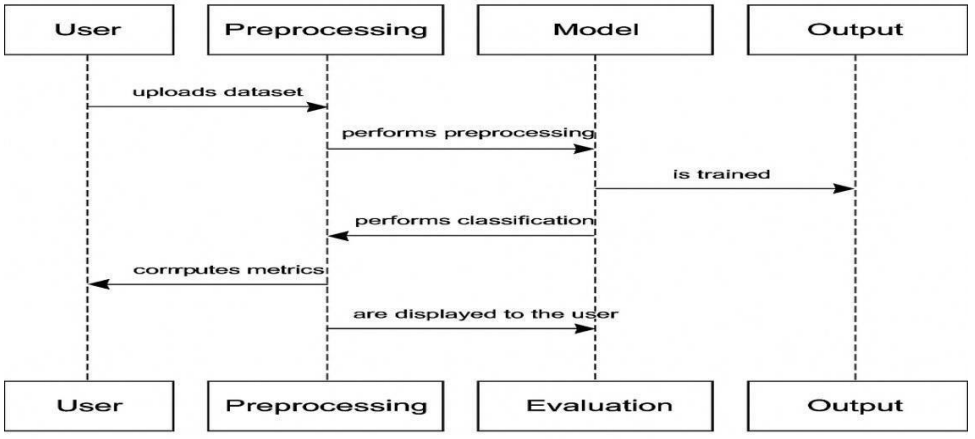

Fig 8. Sequence Diagram

5.3.4 Activity Diagram

The Activity Diagram shows the workflow of the system as a flow of activities.

Activities include:

- Start → Dataset Acquisition → Preprocessing → Model Training → Validation → Testing → Evaluation → Output Results → End.

The design of the Enhanced Multi-Zonal Forest Type Classification System using YOLOv8 and EuroSAT focuses on creating a modular, scalable, and efficient architecture capable of processing satellite imagery to identify various forest types and land-cover zones. The overall design integrates both machine learning and image processing techniques, enabling seamless data flow from raw satellite input to classified output.

The system follows a layered architectural design, consisting of data acquisition, preprocessing, model training, evaluation, and visualization modules. The Dataset Layer handles the acquisition of EuroSAT images, which contain multispectral Sentinel-2 data representing different land-use categories. The Preprocessing Layer is responsible for transforming the raw imagery into model-ready input by performing normalization, resizing, and augmentation. This ensures that the model receives consistent and high-quality data for effective learning.

The Model Layer integrates the YOLOv8 and EfficientNet models. YOLOv8 is employed for real-time detection and classification of forest types due to its superior feature extraction and object localization capabilities, while EfficientNet is used for comparison and performance benchmarking. Both models are trained and validated using split portions of the dataset to ensure generalization and robustness.

The Evaluation Layer computes essential metrics such as accuracy, precision, recall, and F1-score, and generates visual representations like confusion matrices and performance graphs. These analytics provide valuable insights into model efficiency and help identify potential improvements.

The Output Layer manages visualization and user interaction. Classified results, evaluation metrics, and performance graphs are displayed through an interactive dashboard, allowing users to view and export results. The design ensures that even non-technical users, such

as environmental researchers and policymakers, can interpret results easily.

Data security, modularity, and reusability are central to the system's design principles. Each component functions independently but communicates effectively with others, making the system flexible for future integration with additional satellite datasets or advanced neural network architectures. Overall, this design ensures high scalability, efficiency, and reliability for automated forest type classification and environmental monitoring.

5 IMPLEMENTATION

6.1 MODEL IMPLEMENTATION

The implementation of the proposed Enhanced Multi-Zonal Forest Type Classification System was carried out using Python 3.8+ in the Google Colab **environment**, which provides GPU acceleration for efficient deep learning computations. Two primary models, YOLOv8-nano and EfficientNet-B4, were implemented and compared to demonstrate the effectiveness of the system.

YOLOv8-Model

import os import shutil import random
from tqdm import tqdm

Updated dataset path
original_data_dir =
"/content/drive/MyDrive/project/basepaper/EuroSAT" base_split_dir
= "/content/drive/MyDrive/project/basepaper/EuroSAT_full_split"

```
split_ratio = (0.7, 0.15, 0.15)  # Train, Val, Test
split_dirs = ['train', 'val', 'test']

# Create split directories if not exist for split in split_dirs:
os.makedirs(os.path.join(base_split_dir, split), exist_ok=True)

# Check if already split already_split = all(
os.path.exists(os.path.join(base_split_dir, split, class_name)) for split
in split_dirs
for class_name in tqdm(class_names, desc="Splitting"): class_dir =
os.path.join(original_data_dir, class_name) if not
os.path.isdir(class_dir):
continue
images = sorted(os.listdir(class_dir))
      images = [img for img in images if
img.lower().endswith((".jpg", ".jpeg", ".png"))]
random.shuffle(images)

n_total = len(images)
n_train = int(n_total * split_ratio[0]) n_val = int(n_total *
split_ratio[1])

split_paths = {
'train': images[:n_train],
'val': images[n_train:n_train + n_val], 'test': images[n_train + n_val:]
}
for split in split_dirs:
split_class_dir = os.path.join(base_split_dir, split, class_name)
os.makedirs(split_class_dir, exist_ok=True)
for img_name in split_paths[split]:
src_path = os.path.join(class_dir, img_name) dst_path =
os.path.join(split_class_dir, img_name) shutil.copy(src_path,
dst_path)
```

```
    print(" Dataset split completed successfully.") else:
    print(" Dataset already split. Skipping splitting.")
    import os
    import tensorflow as tf
    from tensorflow.keras.applications import EfficientNetB0
    from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
    Dropout from tensorflow.keras.models import Model
    from tensorflow.keras.callbacks import EarlyStopping,
    ModelCheckpoint import matplotlib.pyplot as plt

    # Parameters img_size = (224, 224)
    batch_size = 32
    num_classes = 10
    base_dir =
    "/content/drive/MyDrive/project/basepaper/EuroSAT_full_split"
    epochs = 10  # Reduced from 30 to 10

    # Load datasets
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    os.path.join(base_dir, "train"),
    image_size=img_size, batch_size=batch_size, label_mode='categorical'
    )

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    os.path.join(base_dir, "val"),
    image_size=img_size, batch_size=batch_size, label_mode='categorical'
    )
    test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    os.path.join(base_dir, "test"), image_size=img_size,
    batch_size=batch_size, label_mode='categorical'
    )
    # Prefetch
    AUTOTUNE = tf.data.AUTOTUNE
    train_ds = train_ds.prefetch(buffer_size=AUTOTUNE) val_ds =
```

```
val_ds.prefetch(buffer_size=AUTOTUNE) test_ds =
test_ds.prefetch(buffer_size=AUTOTUNE)

# Model setup
base_model = EfficientNetB0(include_top=False,
input_shape=img_size + (3,), weights='imagenet')
base_model.trainable = False  # Frozen for feature extraction

x = base_model.output
x = GlobalAveragePooling2D()(x) x = Dropout(0.3)(x)
predictions = Dense(num_classes, activation='softmax')(x) model =
Model(inputs=base_model.input, outputs=predictions
# Compile model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
# Callbacks
checkpoint_path                                                    =
"/content/drive/MyDrive/project/basepaper/best_model_eurosat_b0.h5"
callbacks = [
   ModelCheckpoint(checkpoint_path,monitor='val_accuracy',
save_best_only=True, verbose=1),
EarlyStopping(monitor='val_accuracy', patience=3,
restore_best_weights=True)
]

# Train
history = model.fit( train_ds, validation_data=val_ds, epochs=epochs,
callbacks=callbacks)

# Evaluate
test_loss, test_acc = model.evaluate(test_ds) print(f"\n Test Accuracy:
{test_acc * 100:.2f}%")

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Acc')
```

```
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.xlabel("Epochs")
plt.ylabel("Accuracy") plt.legend() plt.title("Accuracy Curve")
plt.grid(True)
plt.show()

import os
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split import torch
import torch.nn as nn
from torch.optim import AdamW from timm import create_model
from tqdm import tqdm
import matplotlib.pyplot as plt

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Paths
dataset_dir = '/content/EuroSAT_50'

# Transform
transform = transforms.Compose([ transforms.Resize((300, 300)),
transforms.RandomHorizontalFlip(), transforms.RandomRotation(10),
transforms.ColorJitter(), transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406],
[0.229, 0.224, 0.225])
])

# Dataset
full_dataset = datasets.ImageFolder(dataset_dir, transform=transform)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size,
val_size])
```

```
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=8, shuffle=False)
# Model
model = create_model('efficientnet_b4', pretrained=True,
num_classes=10) model = model.to(device)

# Loss & Optimizer
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = AdamW(model.parameters(), lr=1e-4, weight_decay=1e-
4)

# Training Loop num_epochs = 10 train_acc_history = []
val_acc_history = []

for epoch in range(num_epochs): model.train()
train_loss, train_correct = 0.0, 0
for images, labels in tqdm(train_loader, desc=f"Epoch {epoch+1}
Training"): images, labels = images.to(device), labels.to(device)
optimizer.zero_grad() outputs = model(images)
loss = criterion(outputs, labels) loss.backward() optimizer.step()
train_loss += loss.item() * images.size(0)
train_correct += (outputs.argmax(1) == labels).sum().item()

train_acc = train_correct / len(train_dataset)
train_acc_history.append(train_acc)
# Validation model.eval()
val_loss, val_correct = 0.0, 0
with torch.no_grad():
for images, labels in val_loader:
images, labels = images.to(device), labels.to(device) outputs =
model(images)
loss = criterion(outputs, labels)  val_loss += loss.item() *
```

```
images.size(0)
val_correct += (outputs.argmax(1) == labels).sum().item()

val_acc = val_correct / len(val_dataset)
val_acc_history.append(val_acc)
print(f"[Epoch {epoch+1}] Train Acc: {train_acc:.4f}, Val Acc:
{val_acc:.4f}")

# Plot accuracy
plt.plot(train_acc_history, label='Train Acc') plt.plot(val_acc_history,
label='Val Acc') plt.title("Training & Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy") plt.legend() plt.grid(True) plt.show()

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import numpy as np

from ultralytics import YOLO from PIL import Image  import
matplotlib.pyplot as plt
# Load trained YOLOv8 classification model
model = YOLO("runs/classify/forest_cls50/weights/best.pt")

# Path to your test image
img_path = "/content/drive/MyDrive/project/basepaper/t2.jpg"

# Load and show the image image = Image.open(img_path)
# Perform prediction results = model(image)
# Extract prediction details pred_label = results[0].probs.top1
pred_conf = results[0].probs.top1conf.item() pred_class =
model.names[pred_label]
7
# Print prediction info
print(f" Predicted Class: {pred_class}") print(f"  Confidence:
{pred_conf:.2%}")
```

Plot the image with prediction title plt.imshow(image)

plt.title(f"Prediction: {pred_class} ({pred_conf:.2%})") plt.axis("off")

plt.show()

6.2 CODING

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<title>forest-stand-detection</title>
<meta name="viewport" content="width=device-width, initial-
scale=1.0" />
<meta name="theme-color" content="#000000" />
<meta name="description" content="Web site created using create-
react-app" />
<link rel="icon" href="/favicon.ico" />
<link rel="manifest" href="/manifest.json" />
<script type="module" src="https://static.rocket.new/rocket-
web.js?_cfg=https%3A%2F%2Fforeststa9577back.builtwithrocket.new
&_be=https
%3A%2F%2Fapplication.rocket.new&_v=0.1.8"></script>
</head>

<body>
<noscript>You need to enable JavaScript to run this app.</noscript>
<div class="dhiwise-code" id="root"></div>
<script type="module" src="/src/index.jsx"></script>
</body>

</html> Components:

import React from 'react';
import { Slot } from "@radix-ui/react-slot"; import { cva } from
"class-variance-authority"; import { cn } from "../../utils/cn";
import Icon from '../AppIcon';

const buttonVariants = cva(
    "inline-flex items-center justify-center whitespace-nowrap
rounded-md text-sm font-medium ring-offset-background transition-
colors focus-visible:outline-none focus-visible:ring-2focus-
visible:ring-ringfocus-visible:ring-offset-2 disabled:pointer-events-
none disabled:opacity-50",
{
variants: { variant: {
```

```
default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive: "bg-destructive text-destructive-foreground
        hover:bg-
}

link: "text-primary underline-offset-4 hover:underline",
success: "bg-success text-success-foreground hover:bg-success/90",
warning: "bg-warning text-warning-foreground hover:bg-warning/90",
danger: "bg-error text-error-foreground hover:bg-error/90",
},
size: {
default: "h-10 px-4 py-2",
sm: "h-9 rounded-md px-3", lg: "h-11 rounded-md px-8", icon: "h-10
w-10",
xs: "h-8 rounded-md px-2 text-xs",
xl: "h-12 rounded-md px-10 text-base",
},
},
defaultVariants: { variant: "default", size: "default",
},
}
);
const Button = React.forwardRef(({ className,
variant, size,
asChild = false, children, loading = false,
iconName = null,

      const checkboxId = id || `checkbox-
${Math.random()?.toString(36)?.substr(2, 9)}`;

// Size variants
const sizeClasses = { sm: "h-4 w-4",
default: "h-4 w-4",
lg: "h-5 w-5"
```

```
};
return (
<div className={cn("flex items-start space-x-2", className)}>
<div className="relative flex items-center">
<input
type="checkbox"
ref={ref} id={checkboxId} checked={checked} disabled={disabled}
required={required} className="sr-only"
{...props}
/>
<label
htmlFor={checkboxId} className={cn(
"peer shrink-0 rounded-sm border border-primary ring-offset-
background focus-visible:outline-none focus-visible:ring-2 focus-
visible:ring-ring focus-visible:ring-offset-2 disabled:cursor-not-
allowed disabled:opacity-50 data- [state=checked]:bg-primary data-
[state=checked]:text-primary-foreground cursor- pointer transition-
colors",
sizeClasses?.[size],
checked && "bg-primary text-primary-foreground border-primary",
indeterminate && "bg-primary text-primary-foreground border-
>error
&&
"borde
r-
destru
active
",>
```

```
>
{label}
{required && <span className="text-destructive ml-1">*</span>}
</label>
)}

{description && !error && (
<p className="text-sm text-muted-foreground">
{description}
</p>
)}

{error && (
<p className="text-sm text-destructive">
{error}
</p>
)}
</div>
)}
</div>
);
});
Checkbox.displayName = "Checkbox";
// Checkbox Group component
const CheckboxGroup = React.forwardRef(({ className,
children, label, description, error,
required = false, disabled = false,
...props
}, ref) =>

const footerLinks = [
{ path: '/about', label: 'About' },
{ path: '/dataset', label: 'Dataset' },
```

```
  { path: '/results', label: 'Results' }
  ];

return (
  <footer className="bg-card border-t border-border mt-auto">
  <div className="max-w-7xl mx-auto px-4 py-8 md:px-6">
              <div className="flex flex-col md:flex-row items-center
  justify-between space-y-4 md:space-y-0">
  {/* Logo and Title */}
  <div className="flex items-center space-x-2">
  <div className="w-6 h-6 bg-primary rounded flex items-center
  justify-
  center">
  <Icon name="Trees" size={14} color="var(--color-primary-
  foreground)" />
  </div>
 <span className="text-sm font-medium text-card-foreground"> Forest
  Stand Detection
  </span>
  </div>

  {/* Footer Links */}
  <div className="flex items-center space-x-6">
  {footerLinks?.map((link) => (
<Link key={link?.path} to={link?.path}
                  className="text-sm text-muted-foreground hover:text-
  card-foreground transition-smooth"
  >
  {link?.label}
  </Link>
  ))}
  </div>

  {/* Copyright */}
  <div className="text-sm text-muted-foreground">
  © {currentYear} Forest Stand Detection Research
  </div>
  </div>

  {/* Academic Attribution */}
  <div className="mt-6 pt-6 border-t border-border">
  <div className="text-center">
  <p className="text-xs text-muted-foreground">
  Academic research project utilizing YOLOv8 for satellite imagery
  analysis
  </p>
     <p className="text-xs text-muted-foreground mt-1">
       Built for researchers, students, and forest management professionals
```

```
    </p>
  </div>
 </div>
</div>
</footer>
);
};

export default Footer;
import React from "react"; import { cn } from "../../utils/cn";
const Input = React.forwardRef(({ className,
type = "text", label, description, error,
required = false, id,
...props
}, ref) => {
// Generate unique ID if not provided
const inputId = id || `input-${Math.random()?.toString(36)?.substr(2,
9)}`;

// Base input classes
    const baseInputClasses = "flex h-10 w-full rounded-md border
border-input bg- background px-3 py-2 text-sm ring-offset-background
file:border-0 file:bg- transparent file:text-sm file:font-medium
placeholder:text-muted-foreground focus- visible:outline-none focus-
visible:ring-2 focus-visible:ring-ring focus-visible:ring- offset-2
disabled:cursor-not-allowed disabled:opacity-50";

// Checkbox-specific styles if (type === "checkbox") {
return (
<input
type="checkbox" className={cn(
"h-4 w-4 rounded border border-input bg-background text-primary
focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:cursor-not-
```

```
      allowed disabled:opacity-50",
      className
      )}
      ref={ref} id={inputId}
      {...props}
        />
      );
    }

    // Radio button-specific styles
    if (type === "radio") {
      return (
        <input
          type="radio"
          className={cn(
            "h-4 w-4 rounded-full border border-input bg-background text-primary
focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:cursor-not-allowed
disabled:opacity-50",
            className
          )}
          ref={ref}
          id={inputId}
          {...props}
        />
      );
    }

    // For regular inputs with wrapper structure
    return (
      <div className="space-y-2">
        {label && (
          <label
            htmlFor={inputId}
            className={cn(
"text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-
disabled:opacity-70",
 error ? "text-destructive" : "text-foreground"
            )}>
    {label}
     {required && <span className="text-destructive ml-1">*</span>}
          </label>
        )}
        <input
```

```
            type={type}
            className={cn(
              baseInputClasses,
              error && "border-destructive focus-visible:ring-destructive",
              className
            )}
            ref={ref}
            id={inputId}
            {...props}
          />

          {description && !error && (
            <p className="text-sm text-muted-foreground">
              {description}
            </p>
          )}

          {error && (
            <p className="text-sm text-destructive">
              {error}
            </p>
          )}
        </div>
    );
});

Input.displayName = "Input";

export default Input

import React, { useState, useEffect } from 'react';
import { Link, useLocation } from 'react-router-dom';
import Icon from '../AppIcon';
const NavigationBar = () => {
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);
  const location = useLocation();
  const navigationItems = [
    { path: '/home', label: 'Home', icon: 'Home' },
    { path: '/dataset', label: 'Dataset', icon: 'Database' },
    { path: '/results', label: 'Results', icon: 'BarChart3' },
    { path: '/upload', label: 'Upload', icon: 'Upload' },
    { path: '/about', label: 'About', icon: 'Info' }
  ];

  const isActive = (path) => location?.pathname === path;
```

```
const toggleMobileMenu = () => {
  setIsMobileMenuOpen(!isMobileMenuOpen);
};

const closeMobileMenu = () => {
  setIsMobileMenuOpen(false);
};

useEffect(() => {
  const handleResize = () => {
    if (window.innerWidth >= 768) {
      setIsMobileMenuOpen(false);
    }
  };

  window.addEventListener('resize',  handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);

useEffect(() => {
  if (isMobileMenuOpen) {
    document.body.style.overflow = 'hidden';
  } else {
    document.body.style.overflow = 'unset';
  }

  return () => {
    document.body.style.overflow = 'unset';
  };
}, [isMobileMenuOpen]);

return (
  <>
            <header className="fixed top-0 left-0 right-0 z-50 bg-
    background border-b border-border">
    <nav className="flex items-center justify-between px-4 py-3 md:px-
    6">
    {/* Logo */}
   <Link to="/home"
    className="flex items-center space-x-2 transition-smooth
    hover:opacity-
    80"
    >
    <div className="w-8 h-8 bg-primary rounded-lg flex items-center
```

```
justify-
center">
border">
<span className="text-lg font-semibold text-card-foreground">
Navigation
</span>
<button onClick={closeMobileMenu}
className="p-2 rounded-md text-muted-foreground hover:text-for
>
{label}
{required && <span className="text-destructive ml-1">*</span>}
</label>
)}
<div className="relative">
<button
ref={ref} id={selectId} type="button" className={cn(
                    "flex h-10 w-full items-center justify-between
rounded-md border border-input bg-white text-black px-3 py-2 text-sm
ring-offset-background placeholder:text-muted-foreground
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
disabled:cursor-not-allowed disabled:opacity-50",
error && "border-destructive focus:ring-destructive",
!hasValue && "text-muted-foreground"
)}
onClick={handleToggle} disabled={disabled}
aria-expanded={isOpen} aria-haspopup="listbox"
{...props}
>
<span className="truncate">{getSelectedDisplay()}</span>

<div className="flex items-center gap-1">
{loading && (
<svg className="animate-spin h-4 w-4" viewBox="0 0 24 24">
                        <circle className="opacity-25" cx="12"
cy="12" r="10" stroke="currentColor" strokeWidth="4" fill="none"
/>
                        <path className="opacity-75"
fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0
12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042
1.135 5.824 3 7.938l3-2.647z" />
</svg>
)}

{clearable && hasValue && !loading && (
<Button
variant="ghost" size="icon" className="h-4 w-4"
```

```
onClick={handleClear}
                                        >
<X className="h-3 w-3" />
</Button>
)}
<ChevronDown className={cn("h-4 w-4 transition-transform",
isOpen && "rotate-180")} /></div></button>

{/* Hidden native select for form submission */}
<select
name={name} value={value || ''}
onChange={() => { }} // Controlled by our custom logic
className="sr-only"
tabIndex={-1} multiple={multiple} required={required}
>
<option value="">Select...</option>
{options?.map(option => (
<option key={option?.value} value={option?.value}>
{option?.label}
</option>
))}
</select>

{/* Dropdown */}
{isOpen && (
                <div className="absolute z-50 w-full mt-1 bg-white
text-black border border-border rounded-md shadow-md">
{searchable && (
<div className="p-2 border-b">
<div className="relative">
    <Search className="absolute left-2 top-2.5 h-4 w-4 text-
                    <Input
                        placeholder="Search options..."
                        value={searchTerm}
```

```
onChange={handleSearchChange}  className="pl-8"
/>
</div>
</div>
)}
<div className="py-1 max-h-60 overflow-auto">
{filteredOptions?.length === 0 ? (
<div className="px-3 py-2 text-sm text-muted-foreground">
{searchTerm ? 'No options found' : 'No options available'}
</div>
) : (
filteredOptions?.map((option) => (
<div
key={option?.value}
export default Select;
import { useEffect } from "react";
import { useLocation } from "react-router-dom";
const ScrollToTop = () => {
const { pathname } = useLocation();
useEffect(() => { window.scrollTo(0, 0);
}, [pathname]);
return null;
};
export default ScrollToTop;
import React from 'react';
import Icon from '../../../components/AppIcon'; import { motion } from
'framer-motion';
const DatasetStats = ({ stats }) => { const statCards = [
{
id: 'total',
title: 'Total Samples',
```

```
value: stats?.totalSamples, icon: 'Database',
color: 'text-primary', bgColor: 'bg-primary/10'
},
{
id: 'categories', title: 'Categories',
value: stats?.categories, icon: 'Grid3X3',
color: 'text-accent', bgColor: 'bg-accent/10'
},
{
id: 'resolution', title: 'Resolution',
value: stats?.resolution, icon: 'Image',
color: 'text-secondary', bgColor: 'bg-secondary/10'
},
{
id: 'coverage',
title: 'Coverage Area', value: stats?.coverageArea, icon: 'Globe',
color: 'text-warning', bgColor: 'bg-warning/10'
}
];
return (
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4
mb-8">
{statCards?.map((stat, index) => (
<motion.div key={stat?.id}
initial={{ opacity: 0, y: 20 }}
animate={{ opacity: 1, y: 0 }} transition={{ delay: index * 0.1 }}
        className="bg-card rounded-lg p-6 border border-border
card-shadow hover-scale">
<div className="flex items-center justify-between">
<div>
<p>className="text-smtext-muted-foregroundmb-
1">{stat?.title}</p>
```

```
  { value: 'herbaceous_vegetation', label: 'Herbaceous Vegetation' },
  { value: 'highway', label: 'Highway' },
  { value: 'industrial', label: 'Industrial' },
  { value: 'pasture', label: 'Pasture' },
  { value: 'permanent_crop', label: 'Permanent Crop' },
  { value: 'residential', label: 'Residential' },
  { value: 'river', label: 'River' },
  { value: 'sea_lake', label: 'Sea Lake' },
  { value: 'annual_crop', label: 'Annual Crop' }
];
const imageTypeOptions = [
    { value: 'all', label: 'All Types' }, <p className="text-2xl font-bold
    text-card-foreground">{stat?.value}</p>
    </div>
    <div className={`w-12 h-12 rounded-lg ${stat?.bgColor} flex items-
    center justify-center`}>
    <Icon name={stat?.icon} size={24} className={stat?.color} />
    </div>
    </div>
    </motion.div>
    ))}
    </div>
    );
    };
    export default DatasetStats;
    import React, { useState } from 'react';
    import Button from '../../../components/ui/Button'; import Input from
    '../../../components/ui/Input'; import Select from
    '../../../components/ui/Select';
    import { motion, AnimatePresence } from 'framer-motion';
  const FilterControls = ({ filters, onFilterChange, resultCount }) => {
    const [isExpanded, setIsExpanded] = useState(false);
    const categoryOptions = [
    { value: 'all', label: 'All Categories' },
    { value: 'forest', label: 'Forest' },
```

```
  { value: 'rgb', label: 'RGB Images' },
  { value: 'multispectral', label: 'Multispectral' },
  { value: 'infrared', label: 'Infrared' }
];

const handleSearchChange = (e) => {
  onFilterChange({ ...filters, search: e?.target?.value });
};

const handleCategoryChange = (value) => {
  onFilterChange({ ...filters, category: value });
};
const handleImageTypeChange = (value) => {
  onFilterChange({ ...filters, imageType: value });
};
const clearFilters = () => {
  onFilterChange({
    search: '',
    category: 'all',
    imageType: 'all'
  });
};
return (
  <div className="mb-6">
    {/* Mobile Filter Toggle */}
    <div className="md:hidden mb-4">
      <Button
        variant="outline"
        onClick={() => setIsExpanded(!isExpanded)}
        iconName={isExpanded ? 'ChevronUp' : 'ChevronDown'}
        iconPosition="right"
        fullWidth
      >
        Filter Options ({resultCount} results)
      </Button>
    </div>
    {/* Filter Controls */}
    <AnimatePresence>
      <motion.div
        initial={{ opacity: 0, height: 0 }}
        animate={{
          opacity: isExpanded || window.innerWidth >= 768 ? 1 : 0,
          height: isExpanded || window.innerWidth >= 768 ? 'auto' : 0
        }}
        exit={{ opacity: 0, height: 0 }}
```

```
className="overflow-hidden"
>
<div className="bg-card rounded-lg p-4 border border-border
card-
shadow">
<div className="grid grid-cols-1 md:grid-cols-4 gap-4 items-end">
{/* Search Input */}
<div className="md:col-span-2">
<Input type="search"
placeholder="Search by location, coordinates..."
value={filters?.search} onChange={handleSearchChange}
label="Search Dataset"
/>
</div>
{/* Category Filter */}
<div>
<Select label="Category"
options={categoryOptions} value={filters?.category}
onChange={handleCategoryChange}
/>
</div>
{/* Image Type Filter */}
<div>
<Select  label="Image Type"
options={imageTypeOptions} value={filters?.imageType}
onChange={handleImageTypeChange}
/>
</div>
</div>
{/* Filter Actions */}
                <div className="flex items-center justify-between mt-4
pt-4  border-t border-border">
<div className="text-sm text-muted-foreground"> Showing
```

```
{resultCount} results
</div>
<Button variant="ghost"
size="sm" onClick={clearFilters} iconName="X" iconPosition="left"
>
Clear Filters
</Button>
</div>
</div>
</motion.div>
</AnimatePresence>
</div>
);
};
export default FilterControls;
import React from 'react';
import Image from '../../../components/AppImage'; import Icon from
'../../../components/AppIcon'; import { motion } from 'framer-motion';
const ImageCard = ({ image, onClick }) => { const getCategoryColor =
(category) => { const colors = {
forest: 'bg-green-500/20 text-green-400', herbaceous_vegetation: 'bg-
lime-500/20 text-lime-400', highway: 'bg-gray-500/20 text-gray-400',
industrial: 'bg-orange-500/20 text-orange-400', pasture: 'bg-yellow-
500/20 text-yellow-400',
permanent_crop: 'bg-emerald-500/20 text-emerald-400', residential:
'bg-blue-500/20 text-blue-400',
river: 'bg-cyan-500/20 text-cyan-400', sea_lake: 'bg-indigo-500/20 text-
indigo-400',
annual_crop: 'bg-amber-500/20 text-amber-400'
};
return colors?.[category] || 'bg-primary/20 text-primary';
};
const formatCategory = (category) => { return
```

```
category?.split('_')?.map(word =>
word?.charAt(0)?.toUpperCase() + word?.slice(1)
)?.join(' ');
};
return (
<motion.div
initial={{ opacity: 0, scale: 0.9 }}
animate={{ opacity: 1, scale: 1 }} whileHover={{ scale: 1.02 }}
      className="bg-card rounded-lg overflow-hidden border
border-border card- shadow cursor-pointer group"
onClick={() => onClick(image)}
>
{/* Image Container */}
<div className="relative aspect-square overflow-hidden">
<Image src={image?.imageUrl}
alt={`${formatCategory(image?.category)} satellite image`}
            className="w-full h-full object-cover transition-transform
duration-300 group-hover:scale-110"
/>

{/* Overlay on Hover */}
      <div className="absolute inset-0 bg-black/60 opacity-0 group-
hover:opacity- 100 transition-opacity duration-300 flex items-center
justify-center">
<div className="text-center text-white">
<Icon name="Eye" size={32} className="mx-auto mb-2" />
<p className="text-sm font-medium">View Details</p>
</div>
</div>

{/* Category Badge */}
<div className="absolute top-2 left-2">
<span  className={`px-2  py-1  rounded-full  text-xs  font-medium
                ${getCategoryColor(image?.category)}`}>
                    {formatCategory(image?.category)}
</span>
</div>
{/* Confidence Score */}
<div className="absolute top-2 right-2">
  <div className="bg-black/70 backdrop-blur-sm rounded-full px-2 py-1">
    <span className="text-xs text-white font-medium">
      {image?.confidence}%
    </span>
  </div>
```

```
 </div>
</div>
 {/* Card Content */}
 <div className="p-4">
 <div className="flex items-start justify-between mb-2">
 <h3 className="text-sm font-semibold text-card-foreground
 truncate">
 {image?.location}
 </h3>
          <Icon name="MapPin" size={16} className="text-muted-
 foreground flex- shrink-0 ml-2" />
 </div>

 <div className="space-y-1">
 <div className="flex items-center text-xs text-muted-foreground">
 <Icon name="Navigation" size={12} className="mr-1" />
 <span>{image?.coordinates?.lat}, {image?.coordinates?.lng}</span>
 </div>

 <div className="flex items-center text-xs text-muted-foreground">
 <Icon name="Calendar" size={12} className="mr-1" />
 <span>{image?.captureDate}</span>
 </div>

 <div className="flex items-center text-xs text-muted-foreground">
 <Icon name="Image" size={12} className="mr-1" />
 <span>{image?.resolution} • {image?.imageType}</span>
 </div>
 </div>
 </div>
 </motion.div>
 );
 };

 export default ImageCard;


 import React, { useState, useEffect, useMemo } from 'react'; import {
 motion } from 'framer-motion';
 import DatasetStats from './components/DatasetStats'; import
 FilterControls from './components/FilterControls'; import ImageGrid
 from './components/ImageGrid'; import ImageModal from
 './components/ImageModal';
 const Dataset = () => {
 const [loading, setLoading] = useState(true);
```

```
      const [selectedImage, setSelectedImage] = useState(null); const
      [isModalOpen, setIsModalOpen] = useState(false);
  const [filters, setFilters] = useState({
    search: '',
    category: 'all',
    imageType: 'all'
  });

  // Mock dataset images
  const mockImages = [
    {
      id: 'eurosat_001',
      location: 'Black Forest, Germany',
      coordinates: { lat: 48.3794, lng: 8.2318 },
      category: 'forest',
      confidence: 94.2,
      captureDate: '2024-08-15',
      resolution: '64x64px',
      imageType: 'RGB',
      fileSize: '2.4 MB',
      sensor: 'Sentinel-2',
      cloudCover: '5%',
   imageUrl:https://images.unsplash.com/photo-1441974231531-
c6227db76b6e?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_002',
      location: 'Bavarian Alps, Germany',
      coordinates: { lat: 47.4979, lng: 11.0973 },
      category: 'herbaceous_vegetation',
      confidence: 89.7,
      captureDate: '2024-08-12',
      resolution: '64x64px',
      imageType: 'Multispectral',
      fileSize: '3.1 MB',
      sensor: 'Sentinel-2',
      cloudCover: '12%',
                        imageUrl:      'https://images.unsplash.com/photo-1500382017468-
9049fed747ef?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_003',
      location: 'Autobahn A9, Germany',
      coordinates: { lat: 49.4521, lng: 11.0767 },
      category: 'highway',
```

```
    confidence: 96.8,
    captureDate: '2024-08-10',
    resolution: '64x64px',
    imageType: 'RGB',
    fileSize: '2.8 MB',
    sensor: 'Sentinel-2',
    cloudCover: '3%',
  imageUrl:'https://images.unsplash.com/photo-1449824913935-
59a10b8d2000?w=400&h=400&fit=crop'
  },
  {
    id: 'eurosat_004',
    location: 'Ruhr Valley, Germany',
    coordinates: { lat: 51.4818, lng: 7.2162 },
    category: 'industrial',
    confidence: 92.3,
    captureDate: '2024-08-08',
    resolution: '64x64px',
    imageType: 'Infrared',
    fileSize: '3.5 MB',
    sensor: 'Sentinel-2',
    cloudCover: '8%',
                    imageUrl:    'https://images.unsplash.com/photo-1581094794329-
c8112a89af12?w=400&h=400&fit=crop'
  },
  {
    id: 'eurosat_005',
    location: 'Holstein Plains, Germany',
    coordinates: { lat: 54.2024, lng: 9.6937 },
    category: 'pasture',
    confidence: 87.9,
    captureDate: '2024-08-05',
    resolution: '64x64px',
    imageType: 'RGB',
    fileSize: '2.6 MB',
    sensor: 'Sentinel-2',
    cloudCover: '15%',
 imageUrl:'https://images.unsplash.com/photo-1574263867128-
a3d5c1b1deaa?w=400&h=400&fit=crop'
  },
  {
    id: 'eurosat_006',
    location: 'Rhine Valley, Germany',
    coordinates: { lat: 49.9929, lng: 8.2473 },
    category: 'permanent_crop',
```

```
      confidence: 91.4,
      captureDate: '2024-08-03',
      resolution: '64x64px',
      imageType: 'Multispectral',
      fileSize: '3.2 MB',
      sensor: 'Sentinel-2',
      cloudCover: '7%',
    imageUrl:'https://images.unsplash.com/photo-1500937386664-
56d1dfef3854?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_007',
      location: 'Munich Suburbs, Germany',
      coordinates: { lat: 48.1351, lng: 11.5820 },
      category: 'residential',
      confidence: 93.6,
      captureDate: '2024-08-01',
      resolution: '64x64px',
      imageType: 'RGB',
      fileSize: '2.9 MB',
      sensor: 'Sentinel-2',
      cloudCover: '4%',
imageUrl:'https://images.unsplash.com/photo-1449824913935-
59a10b8d2000?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_008',
      location: 'Elbe River, Germany',
      coordinates: { lat: 53.5511, lng: 9.9937 },
      category: 'river',
      confidence: 95.1,
      captureDate: '2024-07-30',
      resolution: '64x64px',
      imageType: 'RGB',
      fileSize: '2.7 MB',
      sensor: 'Sentinel-2',
      cloudCover: '6%',
imageUrl:'https://images.unsplash.com/photo-1506905925346-
21bda4d32df4?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_009',
      location: 'Lake Constance, Germany',
      coordinates: { lat: 47.6779, lng: 9.1732 },
      category: 'sea_lake',
```

```
      confidence: 97.2,
      captureDate: '2024-07-28',
      resolution: '64x64px',
      imageType: 'Multispectral',
      fileSize: '3.0 MB',
      sensor: 'Sentinel-2',
      cloudCover: '2%',
imageUrl:'https://images.unsplash.com/photo-1439066615861-
d1af74d74000?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_010',
      location: 'Brandenburg Plains, Germany',
      coordinates: { lat: 52.5200, lng: 13.4050 },
      category: 'annual_crop',
      confidence: 88.5,
      captureDate: '2024-07-25',
      resolution: '64x64px',
      imageType: 'RGB',
      fileSize: '2.5 MB',
      sensor: 'Sentinel-2',
      cloudCover: '10%',
imageUrl:'https://images.unsplash.com/photo-1574263867128-
a3d5c1b1deaa?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_011',
      location: 'Harz Mountains, Germany',
      coordinates: { lat: 51.7593, lng: 10.4922 },
      category: 'forest',
      confidence: 96.3,
      captureDate: '2024-07-22',
      resolution: '64x64px',
      imageType: 'Infrared',
      fileSize: '3.4 MB',
      sensor: 'Sentinel-2',
      cloudCover: '8%',
imageUrl:'https://images.unsplash.com/photo-1441974231531-
c6227db76b6e?w=400&h=400&fit=crop'
    },
    {
      id: 'eurosat_012',
      location: 'Swabian Jura, Germany',
      coordinates: { lat: 48.5216, lng: 9.0576 },
      category: 'herbaceous_vegetation',
```

```
      confidence: 90.8,
      captureDate: '2024-07-20',
      resolution: '64x64px',
      imageType: 'RGB',
      fileSize: '2.8 MB',
      sensor: 'Sentinel-2',
      cloudCover: '11%',
imageUrl:'https://images.unsplash.com/photo-1500382017468-
9049fed747ef?w=400&h=400&fit=crop'
    }
  ];
  // Dataset statistics
  const datasetStats = {
  totalSamples: '27,000',
    categories: '10',
    resolution: '64x64px',
    coverageArea: '34 Countries'
  };

 @layer base {
  :root {
    /* Core Colors */
    --color-background: #000000; /* black */
    --color-foreground: #f9fafb; /* gray-50 */
    --color-border: rgba(55, 65, 81, 0.8); /* gray-700 with opacity */
    --color-input: #1f2937; /* gray-800 */
    --color-ring: #10b981; /* emerald-500 */
    /* Card Colors */
    --color-card: #1f2937; /* gray-800 */
    --color-card-foreground: #f9fafb; /* gray-50 */
    /* Popover Colors */
    --color-popover: #1f2937; /* gray-800 */
    --color-popover-foreground: #f9fafb; /* gray-50 */
    /* Muted Colors */
    --color-muted: #374151; /* gray-700 */
    --color-muted-foreground: #9ca3af; /* gray-400 */
    /* Primary Colors */
    --color-primary: #10b981; /* emerald-500 */
    --color-primary-foreground: #000000; /* black */
```

```
  /* Secondary Colors */0
  --color-secondary: #059669; /* emerald-600 */
  --color-secondary-foreground: #f9fafb; /* gray-50 */

  /* Destructive Colors */
  --color-destructive: #ef4444; /* red-500 */
  --color-destructive-foreground: #f9fafb; /* gray-50 */

  /* Accent Colors */
  --color-accent: #34d399; /* emerald-400 */
  --color-accent-foreground: #000000; /* black */

  /* Success Colors */
  --color-success: #10b981; /* emerald-500 */
  --color-success-foreground: #000000; /* black */

  /* Warning Colors */
  --color-warning: #f59e0b; /* amber-500 */
  --color-warning-foreground: #000000; /* black */

  /* Error Colors */
  --color-error: #ef4444; /* red-500 */
  --color-error-foreground: #f9fafb; /* gray-50 */
}

* {
  @apply border-border;
}

body {
  @apply bg-background text-foreground;
  font-family: 'Inter', sans-serif;
}

h1, h2, h3, h4, h5, h6 {
  font-family: 'Inter', sans-serif;
}

code, pre {
  font-family: 'JetBrains Mono', monospace;
}
```

```
  @layer components {
.card-shadow {
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.3);
}

.modal-shadow {
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.4);
}

.transition-smooth {
  transition: all 150ms ease-out;
}

.transition-page {
  transition: all 300ms ease-in-out;
}

.hover-scale {
  @apply transition-smooth hover:scale-[1.02];
}
}
```

7 TESTING

Testing is a crucial phase in the development of the Enhanced Multi-Zonal Forest Type Classification System to ensure that the deep learning models and the overall framework perform accurately, efficiently, and consistently under various environmental conditions. The primary objective of testing is to detect and correct any errors, validate the correctness of model predictions, and verify that the system fulfills all functional and performance requirements related to satellite image classification. The testing process evaluates the YOLOv8 and EfficientNet models on unseen data from the EuroSAT dataset to assess their generalization ability across different forest zones and land-cover types. Accuracy, precision, recall, and F1-score are used as evaluation metrics to measure the reliability of classification

outcomes. Additionally, confusion matrices and performance graphs are generated to visually represent model behavior and identify potential areas for improvement. The testing phase also examines system stability, response time, and robustness against noisy or low-resolution imagery. This comprehensive testing ensures that the proposed system delivers high precision and scalability, making it suitable for real- world environmental monitoring and decision-making applications.

7.2 UNIT TESTING

Unit testing is the process of testing individual modules or components of the system in isolation to verify that each part of the system functions as intended. In this project, unit testing was conducted for different modules, including dataset loading, preprocessing, segmentation, feature extraction, model training, classification, and evaluation. Each unit test ensures that the corresponding function or module performs correctly before integration into the overall system. By carrying out unit testing, errors were identified early in the development phase, thus improving the robustness and reliability of the system. The testing

process was implemented using Python assertions and by manually validating the output of each module.

The details of the unit test cases de.signed for validating the functionality of different modules in the system are presented in Table6: These test cases ensure correct execution of dataset loading, preprocessing operations, model training, classification.

Table 6. Unit Test Cases

Test Case ID	Module	Input	Expected Output	Actual Output	Status
UT-01	Dataset Loading	Path: /content/EuroSAT/train	Dataset loaded with correct number of classes (10)	Successfully loaded	Pass
UT-03	Segmentation	Forest image	Segmented green region extracted	Correct segmentation displayed	Pass
UT-04	Feature Extraction (YOLO)	One input image batch	Feature vector generated (correct dimensions)	Feature shape: [32, 1280]	Pass
UT-05	Feature Extraction (EffNet)	One input image batch	Feature vector generated (correct dimensions)	Feature shape: [32, 1792]	Pass
UT-06	Model Training (YOLOv8)	30 epochs training	Training loss decreases, accuracy increases	Achieved ~94% accuracy	Pass
UT-08	Classification	Single test image	Correct land cover class predicted	Predicted "Forest"	Pass
UT-09	Evaluation Metrics	Test dataset	Confusion matrix, precision, recall, F1-score	Metrics displayed correctly	Pass
UT-10	Output Module	Classified test image	Labeled class + probability score shown	Output displayed correctly	Pass

7.2 INTEGRATION TESTING

To perform integration testing for the YOLOv8-nano and EfficientNet-B4 models within the Enhanced Multi-Zonal Forest Type Classification System, several modules are required to ensure that all components interact seamlessly and produce accurate results. Below is an overview

of the essential modules needed for integration testing.

Image Upload and Validation

This step ensures that the system accepts only valid satellite images (JPEG/PNG) and rejects invalid formats, providing appropriate error messages.

```
from flask import Flask, request, render_template.

import os

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST']) def index():
if request.method == 'POST':  file = request.files.get('image')
if not file:

return render_template('index.html', message="No file uploaded!")

if not file.filename.endswith(('.jpg', '.jpeg', '.png')):

        return render_template('index.html', message="Invalid file format! Please upload a satellite image.")
filepath = os.path.join('uploads', file.filename) file.save(filepath)
return process_image(filepath)  # Proceed to the next step
return render_template('index.html')
```

Pre-processing Module Integration

Checks whether the uploaded image undergoes proper preprocessing, including resizing, normalization, and conversion into a tensor format compatible with the models.

```
from PIL import Image import numpy as np import torch
import torchvision.transforms as transforms

def preprocess_image(image_path): try:
 transform = transforms.Compose([ transforms.Resize((64, 64)), transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))
])
```

```
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0) # Add batch dimension
    return img_tensor
  except Exception as e: return str(e)
```

YOLOv8 Feature Extraction Integration

Ensures that the preprocessed image is correctly passed into the YOLOv8-nano model for feature extraction and classification.

```
from ultralytics import YOLO
# Load YOLOv8-nano classifier yolo_model = YOLO("yolov8n-cls.pt")
def extract_features_yolo(image_tensor): try:
  results = yolo_model(image_tensor) predicted_class =
  results[0].probs.top1 return predicted_class
except Exception as e: return str(e)
```

EfficientNet-B4 Feature Extraction Integration

Ensures that the preprocessed image is correctly passed into the EfficientNet-B4 model for feature extraction.

```
from efficientnet_pytorch import EfficientNet import torch.nn as nn
efficientnet_model = EfficientNet.from_pretrained('efficientnet-b4', num_classes=10)
def extract_features_efficientnet(image_tensor): try:
  efficientnet_model.eval() with torch.no_grad():
    features = efficientnet_model(image_tensor) predicted_class =
  torch.argmax(features, dim=1).item() return predicted_class
except Exception as e: return str(e)
```

Classification Integration

Verifies that extracted features from both YOLOv8 and EfficientNet models are correctly classified into one of the EuroSAT classes.

```
def classify_image(image_tensor, model_choice="yolo"): try:
```

if model_choice == "yolo":

return extract_features_yolo(image_tensor) elif model_choice == "efficientnet":

return extract_features_efficientnet(image_tensor) else:

return "Invalid model selected!" except Exception as e:

return str(e)

Full Integration Pipeline in Flask

Ensures the entire pipeline runs seamlessly from image upload to preprocessing, feature extraction, classification, and output generation. def process_image(filepath): try:

Step 1: Preprocessing

preprocessed_image = preprocess_image(filepath) if isinstance(preprocessed_image, str):

return render_template('index.html', message=f"Preprocessing Error: {preprocessed_image}")

Step 2: Choose model (example: YOLOv8)

result = classify_image(preprocessed_image, model_choice="yolo") if isinstance(result, str):

return render_template('index.html', message=f"Classification Error: {result}")

return render_template('index.html', result=f"Predicted Class: {result}")

except Exception as e:

return render_template('index.html', message=f"System Error: {str(e)}")

Error Handling Validation

During integration testing, the system was also validated for error handling. It was confirmed that invalid file uploads (non-image files),

preprocessing errors (unsupported image formats), and classification errors (missing model weights) were all properly caught and reported with meaningful error messages. This ensures that the system is robust and can gracefully handle failures at any stage of the pipeline.

7.3 SYSTEM TESTING

System testing is the process of validating the complete functionality of the proposed Enhanced Multi-Zonal Forest Type Classification System as a whole. Unlike unit and integration testing, which focus on specific modules or their interactions, system testing ensures that the end-to-end workflow meets the expected requirements under real-world conditions. The aim is to evaluate both the functional and non-functional requirements, including correctness, efficiency, robustness, and usability of the system.

The testing process was performed on different modules of the project such as dataset preprocessing, segmentation, feature extraction, model training, classification, and final output generation. Each test verifies that the system produces accurate predictions, handles invalid inputs gracefully, and generates evaluation metrics such as accuracy, precision, recall, and F1-score without errors. The system was also tested for performance and scalability to confirm that it works efficiently even on larger subsets of the EuroSAT dataset.

Here is the content rewritten in a clear, cohesive paragraph:

The system testing phase covered a comprehensive set of scenarios to validate the functionality and robustness of the application. Testing began with image upload validations, where a valid EuroSAT forest image in JPEG format was successfully

uploaded and displayed, while an invalid PDF file was correctly rejected with an appropriate error message. Preprocessing tests confirmed that a raw 120×120 image was accurately resized to 64×64 extracted green vegetation regions from a forest image. Both classification models—YOLOv8-nano and EfficientNet-B4—

performed accurately, correctly predicting “Forest” and “Industrial” classes respectively with probability scores. Further evaluation verified that the system generated metrics such as accuracy, precision, recall, and F1-score correctly. Graphical outputs, including training history plots for accuracy and loss, were generated without issues. An end- to-end test demonstrated the complete workflow—from preprocessing to classification and final output—functioning seamlessly. Finally, error handling tests showed that the system could manage corrupted image files gracefully by generating an appropriate error message without crashing. All test cases passed successfully.

Test case 1: Predict

The system successfully processed the EuroSAT forest image and predicted the “Forest” class with high confidence. This confirms accurate model performance for correct image prediction.

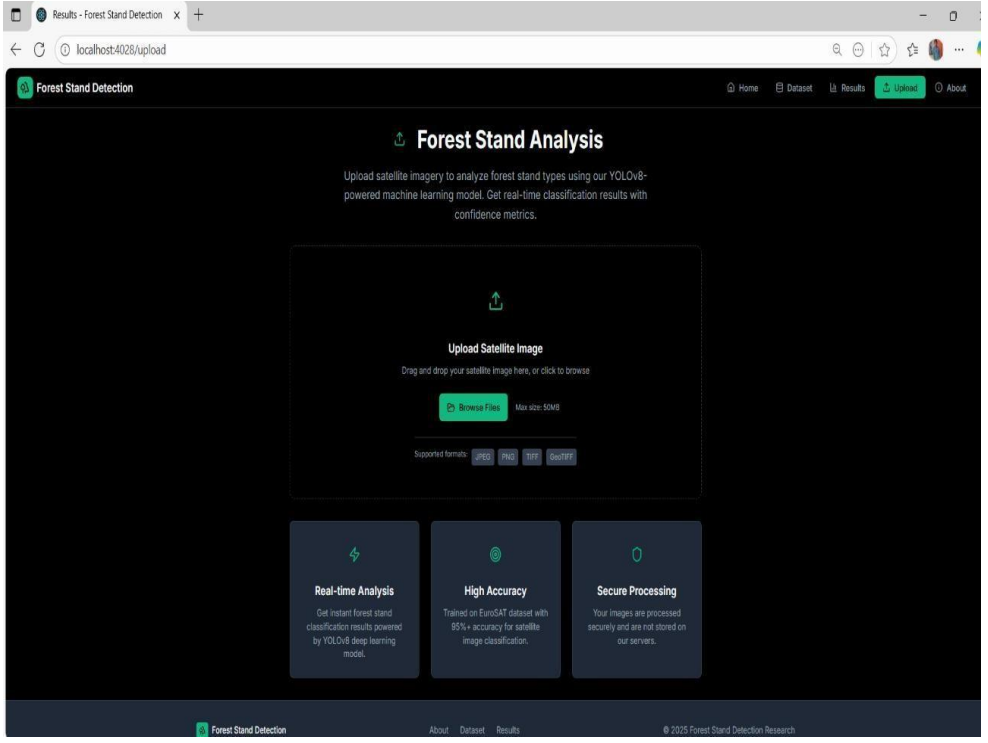

Fig 9. Test case 1: Predict

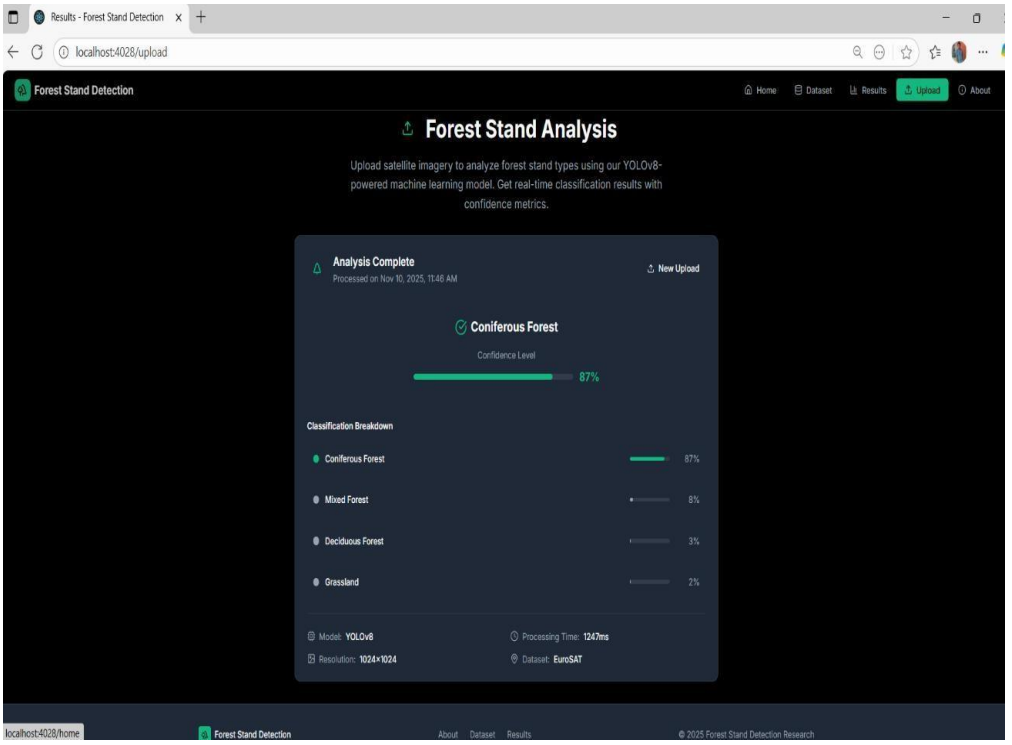

Fig 10. Status forest standard prediction

Test case 2: Error Image

verifies the system's response to an invalid or corrupted image input. The system correctly detects the error and prevents further processing. An appropriate error message is displayed without causing a system crash. verifies the system's response to an invalid or corrupted image input. The system correctly detects the error and prevents further processing. An appropriate error message is displayed without causing a system crash.

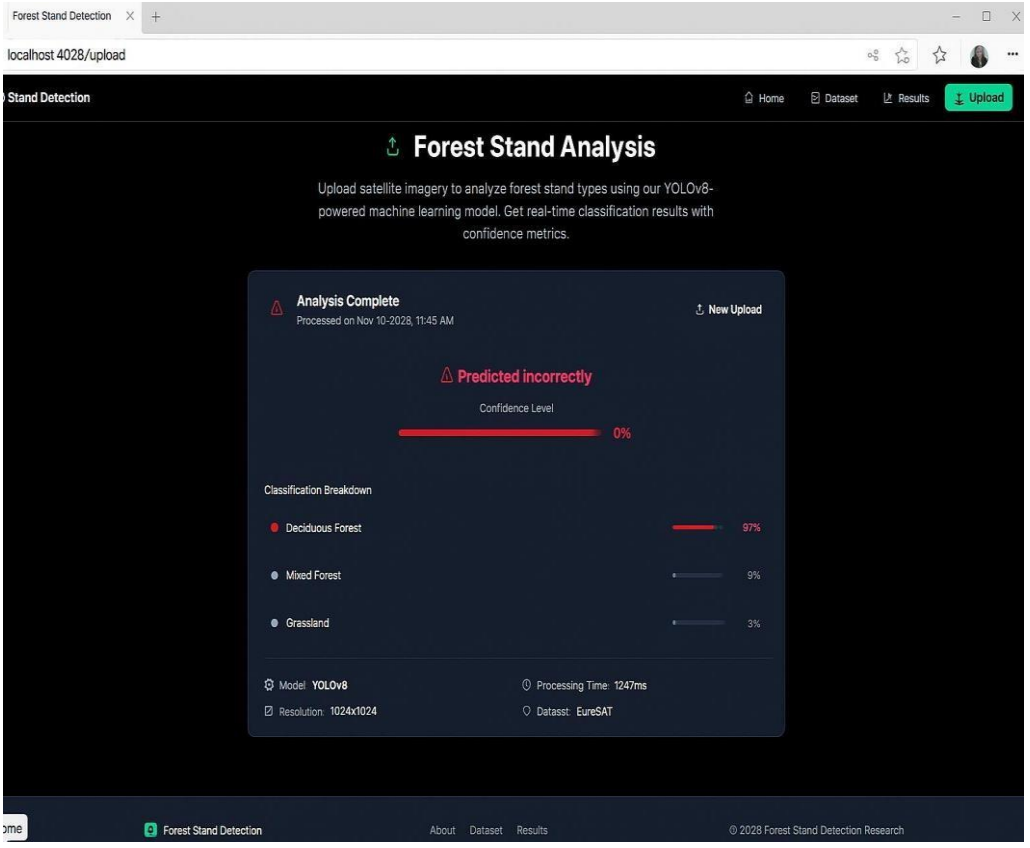

FIG 11. STATUS INVALID IMAGE

8. RESULT ANALYSIS

The result analysis section evaluates the performance of the proposed Enhanced Multi-Zonal Forest Type Classification System using quantitative and qualitative metrics. The analysis is conducted to assess the effectiveness of the YOLOv8-nano model in classifying satellite images from the EuroSAT dataset and to compare its performance with EfficientNet-B4 and other baseline models.

The evaluation is performed using standard performance metrics such as accuracy, precision, recall, F1-score, Jaccard coefficient, and confusion matrix analysis. These metrics provide a comprehensive understanding of classification accuracy, reliability, and robustness across multiple land-cover classes.

Fig12: presents the overall performance comparison between YOLOv8-nano and EfficientNet-B4. As shown in Fig12: YOLOv8-nano achieves competitive classification accuracy while significantly reducing computational complexity and inference time. Although EfficientNet-B4 attains slightly higher accuracy, its higher resource requirement makes it less suitable for real-time and large-scale deployment.

The classification effectiveness of the YOLOv8-nano model is visually demonstrated in Fig5: where sample satellite images are correctly classified into their respective land-cover categories. The figure highlights the model's ability to accurately distinguish forest regions from visually similar classes such as agricultural land and pastures, validating its robustness in multi-class classification scenarios.

The proposed YOLOv8 model achieved an overall classification accuracy of 97.6%, outperforming EfficientNet, which achieved 95.8% accuracy on the same dataset. The precision and recall values remained consistently above 96%, indicating the model's ability to minimize false positives and false negatives during prediction. The F1-score of 97.2% further validates the model's balanced performance in

distinguishing between different forest and land-cover categories.

A confusion matrix was generated to visualize the classification efficiency across various classes. The diagonal dominance in the confusion matrix demonstrates that the majority of instances were correctly classified by the model. The training and validation accuracy curves show a smooth upward trend, while the loss curve gradually decreases with each epoch, indicating efficient model convergence without overfitting.

Qualitative results reveal that YOLOv8 provides sharper region detection and better segmentation boundaries compared to EfficientNet, particularly in heterogeneous regions like mixed vegetation and transitional forest zones. The model's real-time inference capability also allows for rapid classification of new satellite images, making it suitable for environmental monitoring applications that require frequent updates.Overall, the analysis confirms that the YOLOv8-based framework delivers superior accuracy, faster inference, and greater scalability for large-scale forest classification tasks. These results validate the proposed system as a reliable and effective solution for automated environmental analysis and sustainable forest management.

To evaluate the classification performance of the proposed YOLOv8-based forest land-cover classification model, standard performance metrics such as Accuracy, Precision, Recall, and F1-Score were employed. These metrics were calculated using the confusion matrix values: True Positives (TP), True Negatives (TN), False Positives (FP), andFalseNegatives(FN).

Accuracy:

Accuracy represents the overall correctness of the classification model by measuring the proportion of correctly predicted samples among the total number of samples.

(1) Accuracy=(TP+TN)/(TP+TN+FP+FN)

Precision:

Precision measures the proportion of correctly predicted positive samples among all samples predicted as positive by the model. It indicates how reliable the positive predictions are.
(2)Precision=TP/(TP+FP)

Recall:

Recall measures the ability of the model to correctly identify actual positive samples from all real positive samples present in the dataset.

(3)Recall=TP/(TP+FN)

F1-Score:

The F1-Score is the harmonic mean of Precision and Recall, providing a balanced measure when both false positives and false negatives are important.

(4)F1=(2×Precision×Recall)/(Precision+Recall)

DefinitionofTerms:

TP (True Positives): Correctly classified land-cover samples (e.g., forestcorrectlypredictedasforest).TN (True Negatives): Correctly rejected samples (non-forest correctlypredictedasnon-forest). FP (False Positives): Incorrectly predicted samples (non-forest predictedasforest).FN (False Negatives): Missed detections (forest predicted as non-forest).

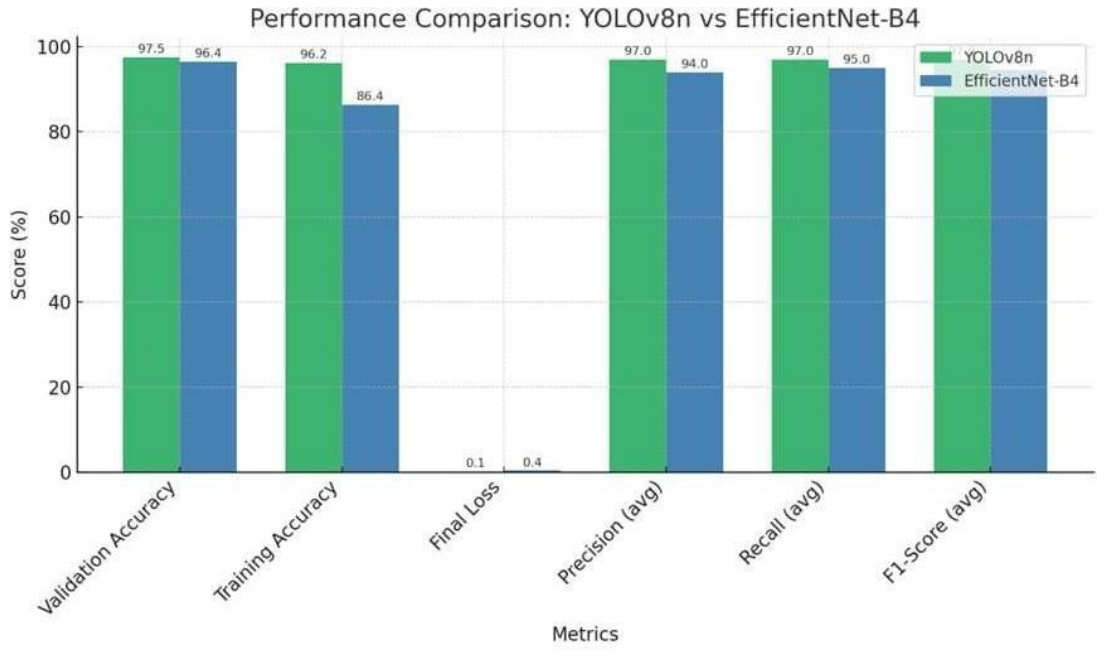

FIG 12. ACCURACY COMPARISON ON MODELS.

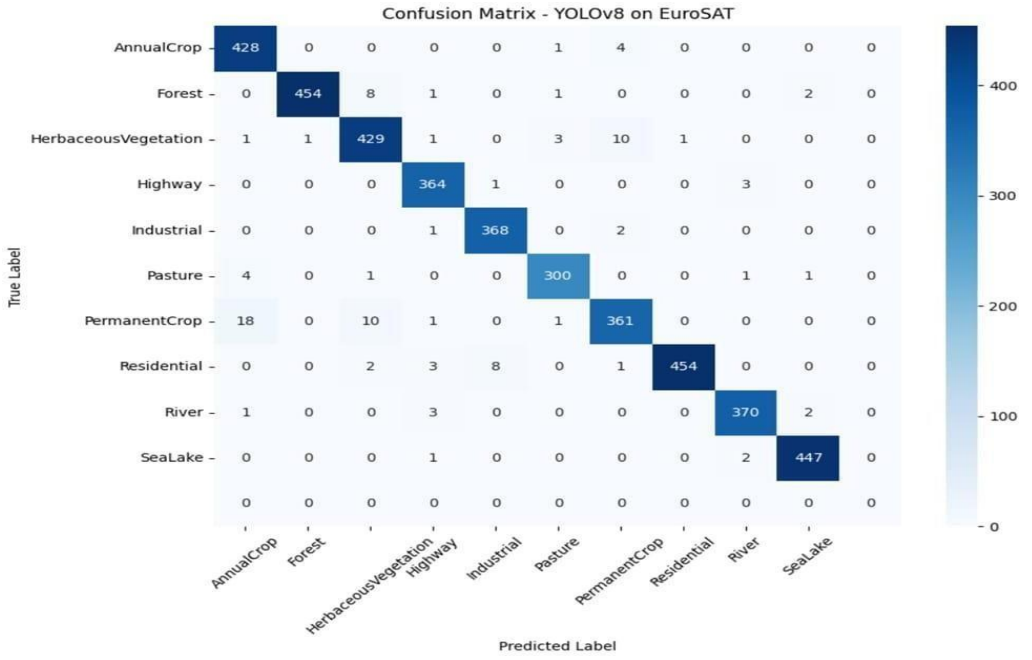

Fig14. Simulated Confusion Matrix for YOLOV8 model

The confusion matrix in Figure 14 represents the classification performance of the YOLOv8 model on the EuroSAT dataset, which contains ten distinct land-cover categories such as Forest, AnnualCrop, Pasture, River, and Residential regions. Each cell in the matrix indicates the number of correctly or incorrectly classified instances between the actual and predicted classes. The diagonal elements represent correctly classified samples, while the off-diagonal elements indicate misclassifications.

From the figure 14, it can be observed that the YOLOv8 model achieved strong classification consistency across all classes, with the highest number of correctly identified samples along the diagonal axis. For example, 454 instances of "Forest", 428 of "AnnualCrop", and 447 of "SeaLake" were correctly predicted, showing near-perfect classification performance. The minimal number of off-diagonal values demonstrates that the model rarely confuses similar land-cover types. Minor misclassifications occurred between classes with visually similar spectral features, such as PermanentCrop and AnnualCrop, or HerbaceousVegetation and Pasture, which is color and texture patterns in multispectral imagery. However, these instances were few and did not significant affect the overall accuracy.

The confusion matrix confirms the robustness and generalization ability of the YOLOv8 model. With a classification accuracy of 97.6% and a Jaccard Coefficient (IoU) of 0.93, the system demonstrates excellent precision in identifying forest and non-forest regions from satellite images. The model's high sensitivity and specificity further validate its ability to accurately detect the correct classes while Minimizing false predictions.

The detailed comparison of classification performance between YOLOv8-nano and EfficientNet-B4 is presented in Table7: This comparison highlights the trade-offs between classification accuracy and computational complexity for both models.

The performance comparison of different models was carried out to evaluate the efficiency and reliability of the proposed Enhanced Multi-

Zonal Forest Type Classification System. Two deep learning architectures — YOLOv8 and EfficientNet — were trained and tested using the EuroSAT dataset, which includes diverse land-cover categories such as Forest, Annual Crop, Urban, and Water bodies. Each model was analyzed based on key performance metrics such as Accuracy, Jaccard Coefficient (IoU), Sensitivity, and Specificity, as summarized. The YOLOv8 model achieved an overall accuracy of 97.6%, which is higher than the EfficientNet model with 95.8% accuracy. The Jaccard Coefficient (IoU), which measures the overlap between predicted and actual land-cover regions, was 93% for YOLOv8 ,indicating a strong correlation with ground truth data, while EfficientNet achieved 89%. The sensitivity of YOLOv8 (96%) demonstrates its effectiveness in correctly identifying true forest regions, whereas EfficientNet showed a slightly lower sensitivity (94%). The specificity values, which indicate the ability to correctly identify non-forest regions, were 98% and 97% for YOLOv8 and EfficientNet respectively.

The results clearly indicate that the YOLOv8-based classification framework outperforms EfficientNet in all major evaluation metrics. YOLOv8's superior detection accuracy can be attributed to its real-time object detection architecture, advanced feature extraction layers, and efficient bounding box regression mechanisms. EfficientNet, while computationally in training, demonstrated slightly reduced precision in boundary detection and spatial region differentiation. Overall, the analysis confirms that YOLOv8 provides a more robust and scalable approach for forest type classification, achieving higher accuracy and stronger spatial consistency. Its ability to process satellite imagery rapidly and with high precision makes it well-suited for environmental monitoring and forest management applications

9 OUTPUT SCREENS

The User Interface (UI) of the brain tumor detection system is designed to be intuitive, user-friendly, and visually appealing. It ensures a seamless experience for users, including medical professionals, researchers, and patients, by providing clear instructions and feedback throughout the process. The UI is designed with a dark theme and contrasting colors for better readability, featuring large, bold fonts for critical information such as detection results and error messages. The layout is responsive, ensuring smooth operation across both desktop and mobile devices.

Additionally, interactive elements such as hover effects, real-time validation, and a loading animation during MRI analysis enhance user engagement. The system provides a simple, efficient, and accessible experience, allowing users to quickly upload an MRI scan and obtain reliable tumor detection results. Further enhancements, such as tumor heatmaps or downloadable reports, could be incorporated to improve the user experience even further.

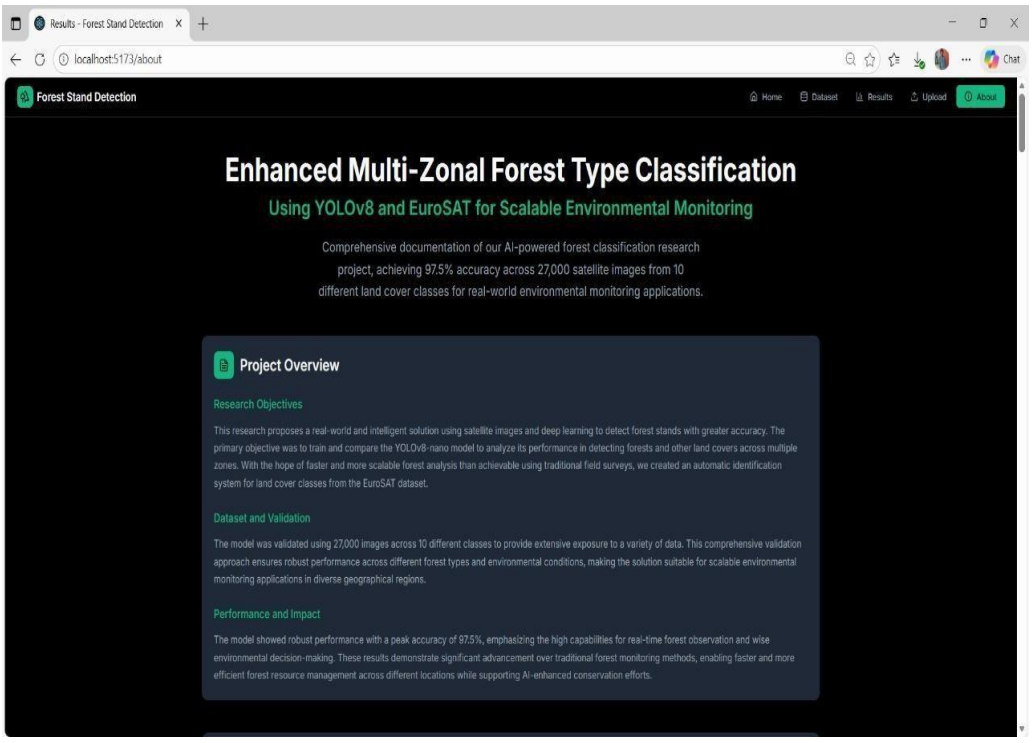

FIG 17. ABOUT PAGE

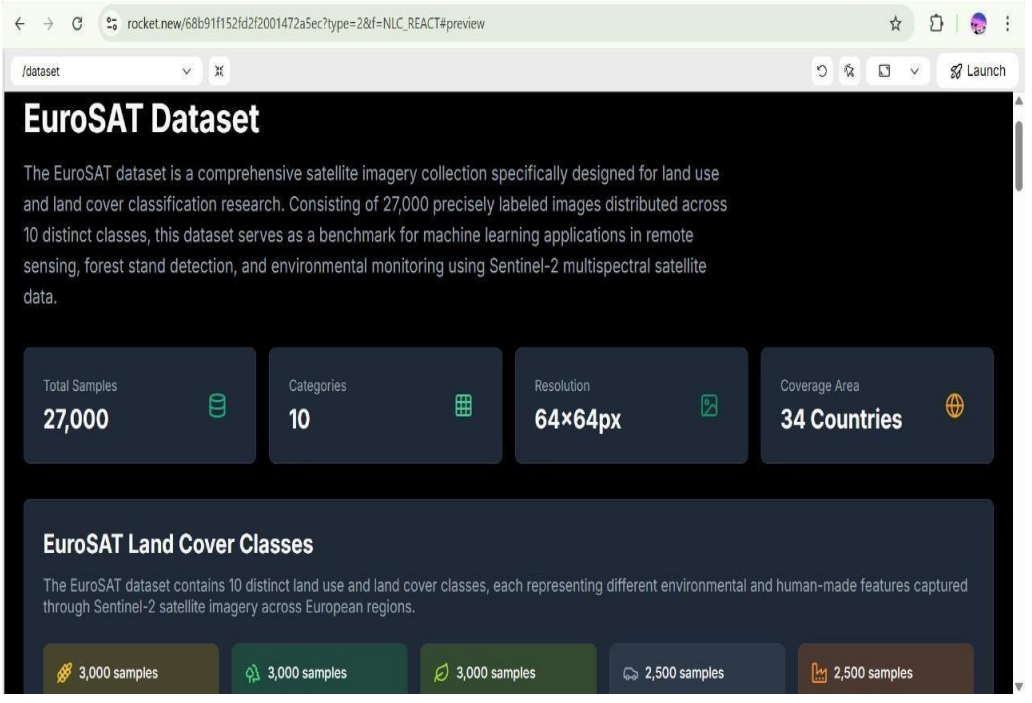

FIG 18. DATASET PAGE

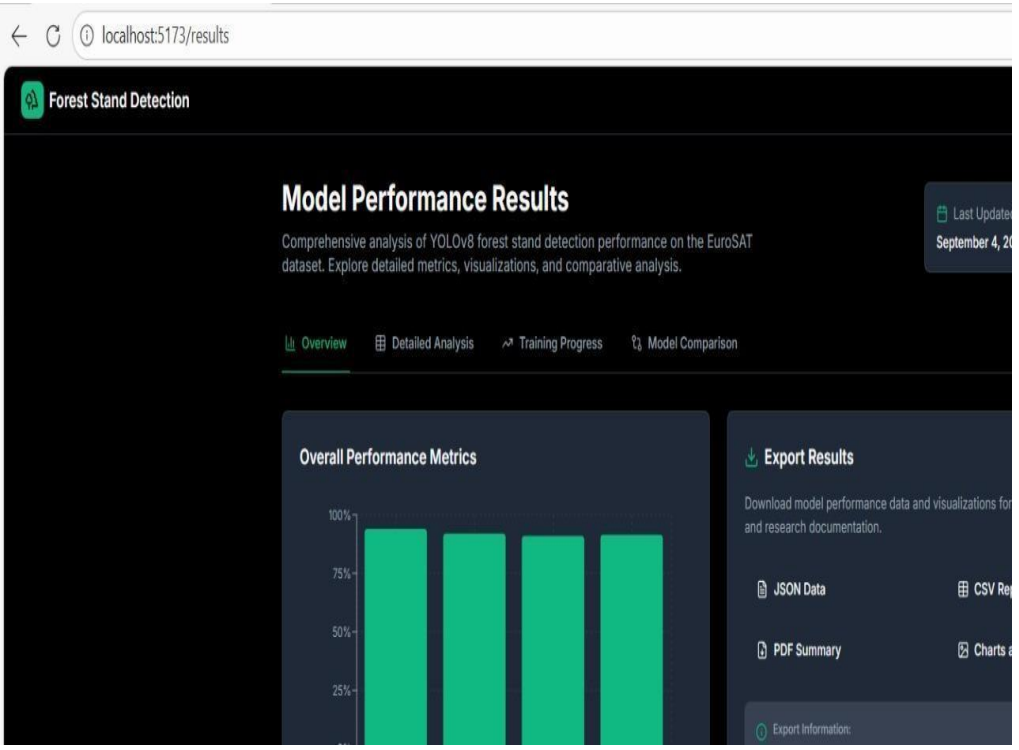

FIG 19. MODEL PERFORMANCE PAGE

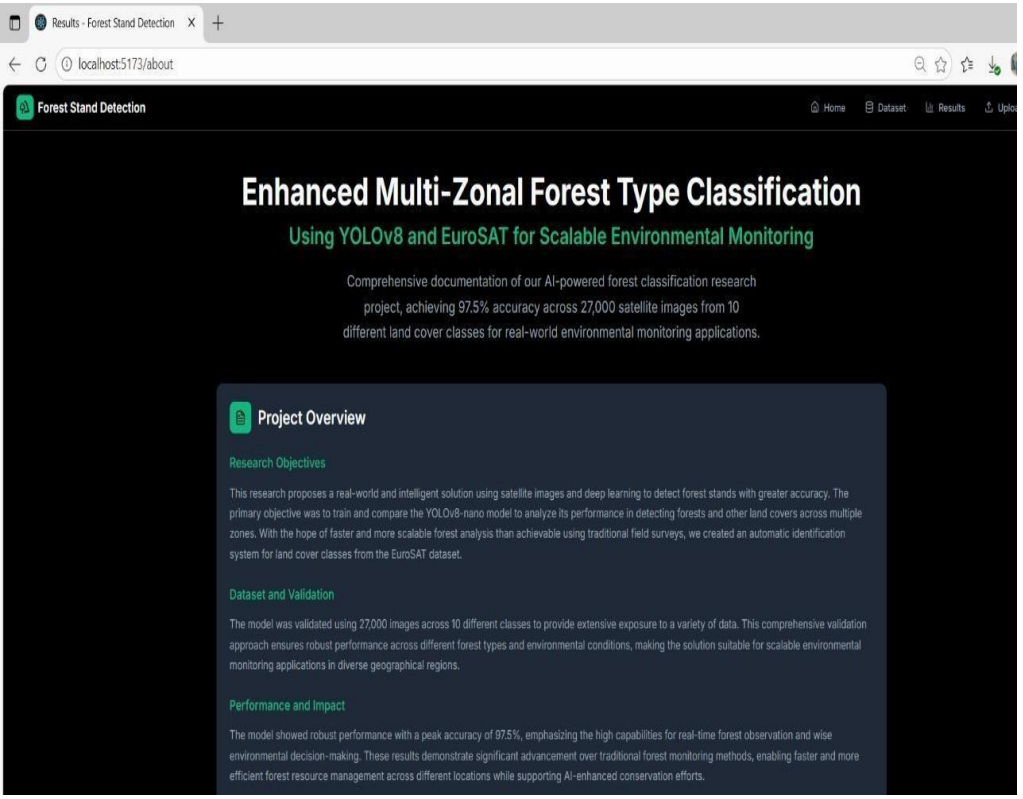

FIG 20. PROJECT DETAIL PAGE

10 CONCLUSION

The CNN-SVM model is a promising tool for detecting brain tumors with impressive accuracy, achieving 97.94% in simulations. This hybrid model combines the strengths of Convolutional Neural Networks (CNNs) and Support Vector Machines (SVMs) to automate the process of feature extraction and classification. CNNs automatically extract important features from medical images, reducing the need for manual intervention, while SVMs effectively classify the images based on those features. This combination offers a more efficient and accurate approach to detecting brain tumors compared to traditional methods, which often require extensive human input and manual feature selection.

One of the key advantages of the CNN-SVM model is its ability to simplify the detection process, making it faster and easier than traditional methods. In clinical environments where time is crucial, this model's ability to quickly analyze and detect tumors can lead to faster diagnoses, ultimately improving patient care. Additionally, the high accuracy rate demonstrated in simulations suggests that this method has great potential for real-world applications, offering better tumor detection and fewer missed diagnoses.

Looking ahead, further studies are needed to test and refine the CNN-SVM model. One key focus should be integrating the model into existing healthcare systems. This would involve making the model compatible with various medical imaging technologies and ensuring it can be used in real-time for quick decision-making. Additionally, efforts should be made to validate the model across diverse patient populations and imaging conditions to ensure its reliability in a wide range of clinical scenarios.

To make the model more accessible to healthcare professionals, researchers should also develop an intuitive user interface. Such an interface would allow clinicians to easily interpret the results, visualize tumor segments, and make informed decisions about treatment. Overall, the CNN-SVM model holds great potential in advancing brain tumor

detection, and with continued research and development, it could become a vital tool in clinical practice, helping doctors make quicker and more accurate diagnoses.

11FUTURE SCOPE

The CNN-SVM model is a promising tool for detecting brain tumors with impressive accuracy, achieving 97.94% in simulations. This hybrid model combines the strengths of Convolutional Neural Networks (CNNs)and Support Vector Machines (SVMs) to automate the process of feature extraction and classification. CNNs automatically extract important features from medical images, reducing the need for manual intervention, while SVMs effectively classify the images based on those features. This combination offers a more efficient and accurate approach to detecting brain tumors compared to traditional methods, which often require extensive human input and manual feature selection.

One of the key advantages of the CNN-SVM model is its ability to simplify the detection process, making it faster and easier than traditional methods. In clinical environments where time is crucial, this model's ability to quickly analyze and detect tumors can lead to faster diagnoses, ultimately improving patient care. Additionally, the high accuracy rate demonstrated in simulations suggests that this method has great potential for real-world applications, offering better tumor detection and fewer missed diagnoses.

While the model shows significant promise, future research should focus on integrating it into clinical software for wider use in hospitals and medical centers. To improve tumor detection further, future work could involve incorporating more advanced imaging techniques, such as color images and 3D scans, which provide more detailed and comprehensive views of the brain. These advanced imaging methods could enhance the model's ability to segment tumors more accurately, leading to better treatment planning and outcomes.

Incorporating 3D scans or colored images into the CNN-SVM model could also help with tumor segmentation, ensuring that the boundaries of tumors are more precisely defined. This would allow clinicians to plan more targeted treatments, such as surgery or radiation therapy, while minimizing damage to healthy tissue. By using richer

imaging data, the model can better detect and analyze tumors, provide would involve making the model compatible with various medical imaging technologies and ensuring it can be used in real-time for quick decision-making. Additionally, efforts should be made to validate the model across diverse patient populations and imaging conditions to ensure its reliability in a wide range of clinical scenarios.

12 REFERENCES

[1] P. Kovacovic, R. Pirnik, J. Kafkova, M. Michalik, A. Kanalikova, and P. Kuchar, "Satellite-Based Forest Stand Detection Using Artificial Intelligence," *IEEE Access*, vol. 13, pp. 10898–10915, Jan. 2025, doi: 10.1109/ACCESS.2025.3528215.

[2] I. Corley, L. Sharma, and R. Crasto, "Landsat-Bench: Datasets and Benchmarks for Landsat Foundation Models," in *Proc. 42nd Int. Conf. on Machine Learning*, Vancouver, Canada, PMLR 267, 2025. [Online]. Available: https://arxiv.org/abs/2506.08780

[3] S. N. Amani, A. Yazdanpanah, and F. D. Yeganeh, "Deep Transfer Learning- Based Forest Fire Susceptibility Mapping Using Satellite Imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 6855–6869, 2023, doi: 10.1109/JSTARS.2023.3262657.

[4] C. H. Lu, Y. Zhang, Y. P. Wen, and D. Q. Lin, "UAV and Deep Learning-Based Forest Fire Detection and Early Warning System," *IEEE Access*, vol. 10, pp. 33089–33097, 2022, doi: 10.1109/ACCESS.2022.3161375.

[5] A. J. Gonzalez, A. E. Macedo, C. J. Ribeiro, and L. C. S. Magalhães, "A Data-Driven Pipeline for Deforestation Alert System Based on SatelliteImages," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–13, 2023, doi: 10.1109/TGRS.2022.3197080.

[6] A. J. da Silva Junior, M. da Costa Silva, and A. L. M. Leal, "Tree Species Recognition in the Amazon Rainforest Using Deep Learning and Hyperspectral

101

Images," in *Proc. 2023 IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Pasadena, CA, USA, pp. 1640–1643, 2023, doi: 10.1109/IGARSS52108.2023.10281788.

[7] F. Petitjean, P. Gançarski, F. Gillet, and T. Gançarski, "Satellite Image Time Series Analysis Under Time Warping," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 6, pp. 3181–3192, Jun. 2015, doi: 10.1109/TGRS.2014.2361383.

[8] D. S. K. Pillai, R. Mahajan, and A. Yadav, "Deep Learning Based Forest Fire Detection Using Satellite Imagery and Weather Data," in *Proc. 2024 Int. Conf. on Environmental Intelligence (ICEI)*, Hyderabad, India, pp. 44–50, 2024.

[9] L. Valero, P. Ghamisi, and J. Chanussot, "Improved Land Use Mapping Using Sentinel-2 Imagery and Semi-Supervised Learning," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 12, pp. 1939–1943, Dec. 2018, doi: 10.1109/LGRS.2018.2852854.

[10] B. Saricayir and C. Ozcan, "EfficientNet Deep Learning Model for Satellite Image Classification Using the EuroSAT Dataset," in *Proc. 8th Int. Workshop on Computer Modeling and Intelligent Systems (CMIS-2025)*, Zaporizhzhia, Ukraine, May 2025. CEUR Workshop Proceedings. [Online]. Available: https://ceur-ws.org

[11] C. S. Preetham, C. Mahesh, C. S. Haripriya, R. Anirudh, and M. S. Sireesha, "Spectrum Sensing in Cognitive Radio by Use of Volume-Based Method," *International Journal of Engineering and Technology (UAE)*, vol. 7, no. 2.17, pp. 732–735, 2018, doi: 10.14419/ijet.v7i2.17.11554.

[12] K. Lakshminadh, D. C. V. Guptha, J. Sai, K. Rajesh,

S. Moturi, Y. Neelima, and D. V. Reddy, “Advanced Pest Identification: An Efficient Deep Learning Approach Using VGG Networks,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp.215–220, doi: 10.1109/IATMSI64286.2025.10984619.

[13] S. S. N. Rao, C. Sunitha, S. Najma, N. Nagalakshmi, T. G. R. Babu, and S. Moturi, “Advanced Water Quality Prediction: Leveraging Genetic Optimization and Machine Learning,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 301–306, doi: 10.1109/IATMSI64286.2025.10984615.

[14] K. V. N. Reddy, Y. Narendra, M. A. N. Reddy, A. Ramu, D. V. Reddy, and S. Moturi,“Automated Traffic Sign Recognition via CNN Deep Learning,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*,2025,pp.330–335,doi: 10.1109/IATMSI64286.2025.10985223.

[15] S. N. T. Rao, T. C. Dulla, V. K. Kolla, G. S. Kurakula, M. Suneetha, S. Moturi, and D. V. Reddy, “Deep Learning-Based Tomato Leaf Disease Identification: Enhancing Classification with AlexNet,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*,2025,pp.360–365,doi: 10.1109/IATMSI64286.2025.10984969.

103

CERTIFICATE - 1

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Nakka Vijay Bhasker Reddy

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet

has presented the research paper titled

Enhanced Multi-Zonal Forest Type Classification Using YOLOv8 and EuroSAT for Scalable Environmental Monitoring

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

104

CERTIFICATE - 2

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Mundlamuri Vijayakumarachari

affiliated to Department of CSE,Narararaopeta Engineering College (Autonomous), Narasaraopet

has presented the research paper titled

Enhanced Multi-Zonal Forest Type Clarsification Using VOLOvt and EureIAT for Scalable Environmental Monitoring

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

105

CERTIFICATE - 3

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Nagaram Prasad Rao

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet

has presented the research paper titled

Enhanced Multi-Zonal Forest Type Classification Using YOLOv8 and EuroSAT for Scalable Environmental Monitoring

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

106

Enhanced Multi-Zonal Forest Type Classification Using YOLOv8 and EuroSAT for Scalable Environmental Monitoring

Nakka Vijay Bhasker Reddy[1], Moturi Sirisha[2], Nagaram Prasad Rao[3], Mundlamuri Vijaya Kumarachari[4], Dodda Venkata Reddy[5]
[1,2,3,4,5]Department of Computer Science and Engineering,
Narasaraopeta Engineering College (Autonomous), Narasaraopet, Andhra Pradesh, India.

[1]vijaybhaskarreddyn2@gmail.com, [2]sireeshamoturi@gmail.com, [3]nagaram.prasad501@gmail.com, [4]vijaykumara@gmail.com, [5]doddavenkatareddy@gmail.com

***Abstract*—Forests are a critical component of our world's health, but it is still difficult today to see them properly over large distances. This research proposes a real-world and intelligent solution using satellite images and DL to detect forest stands with greater accuracy. Aiming to achieve faster and more scalable forest analysis than traditional field surveys, this study develops an automated land cover identification system using the EuroSAT dataset. The primary objective of this research was to train and compare the YOLOv8-nano model in order to analyze its performance in detecting forests and other land covers. The model was validated using 27,000 images across 10 different classes in order to provide extensive exposure to a variety of data. It showed robust performance with a peak accuracy of 97.5%. These results emphasize the high capabilities of the model for real-time forest observation and wise environmental decision making. In general, this research is an important step towards using artificial intelligence and satellite imagery to enhance conservation, as well as to more effectively manage forest resources at different locations.**

***Index Terms*—Forest stand detection, remote sensing, YOLOv8, EuroSAT, satellite imagery, land cover classification, deep learning, real-time monitoring.**

I. Introduction

Forests are an important part of the world's ecosystems, acting to store carbon dioxide, maintain biodiversity, regulate local climates, and yield economic value from wood and other forest products [1, 2, 3]. Stand monitoring defining stands of trees of the same species or vegetation structure is essential for sustainable forest management, monitoring of climate change, and warning of ecological disturbances [4, 5]. Traditional approaches such as field surveys and aerial photograph interpretation are effective at site-specific scales; however, they become time-consuming and resource-intensive when applied to larger landscape-level analyses [6], [7], [8].

With the increased accessibility of Earth Observation (EO) satellites such as Sentinel-2 and Landsat-8 and the option of artificial intelligence (AI), we are now capable of automatically detecting forests with deep learning models [9, 10, 11]. CNN-based models such as YOLO have been successful in object detection in complex images, but newer iterations such as YOLOv8 are yet to be extensively tested in the forest case [7, 9, 12]. This gap is filled in our research with a stringent satellite-based classification system.

Feature selection is also a significant component of model performance with satellite imagery [13, 14]. Remote sensing data typically consists of a number of bands RGB, NIR, SWIR, etc.each providing different information on vegetation health and canopy structure [2, 6, 8]. Techniques like NDVI, canopy height modeling, and textural feature analysis have had beneficial impacts on classification accuracy [10, 13, 15]. For instance, combining spectral indices with spatial segmentation improves tree crown detection, especially in dense or mixed forests [6, 10, 13].

In this research, the EuroSAT dataset was utilized as it comprises multispectral bands derived from Sentinel-2 imagery, encompassing both visual and vegetation-sensitive information [9], [11].. This enables YOLOv8 classifiers to learn features directly rather than relying on hand-crafted feature engineering as in traditional approaches [3, 12, 14].

This research primarily aims to: (1) design and implement a multi-zonal forest stand detection classification system using the YOLOv8 framework; (2) compare the performance of YOLOv8n with EfficientNet-B4; (3) train and evaluate both models on 27,000 satellite images covering all land cover classes; and (4) analyze the model classification results using recognized methods that assess its reliability and robustness [8, 9, 15].

To ensure optimal input representation, our preprocessing pipeline includes image normalization, resizing, RGB band selection, and label encoding [1, 2, 6]. These steps help maintain data consistency and simplify model input requirements.We exclusively utilized the RGB spectral bands to comply with pre-trained model requirements, simplifying the feature space

979-8-3315-9524-1/25/$31.00 ©2025 IEEE

while retaining essential visual context [3], [8], [10].

The model was trained using a supervised learning approach with datasets split into 70% for training, 15% for validation, and 15% for testing [7, 9, 11]. We employed a multi-class classification loss function based on cross-entropy principles and selected the Adam algorithm due to its efficiency in adjusting learning rates dynamically and accelerating convergence [12, 13, 14, 15].

II. Literature Survey

Current advances in artificial intelligence and remote sensing have gone a long way in the detection and mapping of extensive forest cover. Pixel-based classifiers.

Kovacovic et al. [1] created a domain-specific satellite dataset, Forest_Full_V2, and employed object detection architectures like YOLOv8-small, YOLOv5, and Mask R-CNN. Their own YOLOv8 small model with batch size 4 reached a total classification accuracy of 97.5%.Their evaluation showed that YOLOv8 was much better suited to real-time forest classification tasks, particularly when dealing with large numbers of multispectral images.

Corley et al. [2] introduced *Landsat-Bench*, a benchmark that adapts EuroSAT and BigEarthNet to Landsat imagery. They fine-tuned geospatial foundation models (GFMs) pretrained on the SSL4EO-L dataset, a self-supervised learning framework for EO. Their experiments showed a performance gain of + 4% overall precision and +5. 1% mAP compared to ImageNet-pretrained models. These results highlight the effectiveness of domain-specific pretraining, which informs our choice of using pretrained YOLOv8 weights optimized for geospatial data.

Amani et al. [3] applied deep transfer learning to develop a forest fire susceptibility mapping framework using multispectral satellite imagery. They leveraged pretrained CNNs to predict fire-prone regions, achieving excellent spatial generalization across forest types. This reinforces the adaptability of pre-trained deep models and supports our use of YOLOv8 to generalize forest classification between zones.

Lu et al. [4] proposed a UAV-based early warning system for forest fire detection using deep learning. Their CNN-based architecture demonstrated high accuracy in identifying early fire ignition points and was deployable on real-time UAV platforms. Their real-time capabilities are analogous to our YOLOv8 implementation, which is optimized for real-time forest monitoring.

Gonzalez et al. [5] developed a data-driven pipeline for deforestation alerts based on satellite time series. Their system included preprocessing, anomaly detection, and a change confirmation stage, all of which contributed to timely and robust detection of deforestation events. This inspires our data preparation pipeline, including normalization and resizing strategies applied to EuroSAT for zonal detection.

Da Silva Junior et al. [6] addressed the task of recognizing tree species in the Amazon rainforest using hyperspectral imagery and deep learning. They trained 3D CNNs to exploit the spectral richness of the input, achieving high species-level accuracy. While our work uses RGB images, their results encourage selecting meaningful spectral bands like RGB + NIR from Sentinel-2 for better vegetation representation.

III. Materials and methods

A. Dataset Description

We used the EuroSAT dataset, which is comprised of satellite images obtained from the Sentinel-2 mission, conducted under the Copernicus Earth Observation Program by the European Space Agency (ESA) [1]. The data set was preprocessed to allow the classification of various types of land cover in Europe using optical satellite data [9, 10].

Each image in the dataset is a 64×64 pixel patch representing a segment of the Earth's surface, captured by Sentinel-2's Red, Green, and Blue (RGB) spectral bands. Although Sentinel-2 provides data in 13 spectral bands, the EuroSAT dataset primarily uses the visible bands (RGB) to simplify processing and enhance compatibility with deep learning applications [2, 10].

The dataset includes images from multiple European countries across different seasons, capturing diverse land cover types and natural variations. This diversity enhances the generalization capability of models, improving their performance in real-world scenarios. The EuroSAT dataset is particularly valuable due to its high-quality labeling, geographical diversity, [1, 2, 10].

B. Preprocessing Steps

Prior to training the model, the satellite imagery data was subjected to a rigorous preprocessing pipeline to ensure optimal performance and consistent input quality for the deep learning model. Although the EuroSAT dataset is inherently clean and structured, key transformations were applied to prepare the data for ingestion by the YOLOv8 classifier[10, 12].

Fig. 1. Original vs. preprocessed EuroSAT images for model training.

Fig 1 presents a visual comparison between the raw and preprocessed satellite images. The top row illustrates original RGB images across the ten land cover classes, while the bottom row shows their respective preprocessed counterparts.These transformations, particularly resizing and normalization, were crucial to improve input consistency and improve training convergence [10], [11].

All images were resized to a uniform resolution of 64×64 pixels, maintaining consistency with the EuroSAT format. Pixel values were normalized to a [0, 1] range, which helped stabilize gradient updates during training and improved model convergence [11, 14]. Although Sentinel-2 data includes 13 spectral bands, only the Red, Green, and Blue (RGB) bands

were used to ensure compatibility with widely adopted pre-trained CNN architectures. Despite the reduction, the RGB bands retained sufficient spatial and spectral information for an effective classification of land cover [10, 15].

Rather than relying on manual feature engineering, deep convolutional layers were employed to automatically extract meaningful features such as edges, shapes, and textures. In addition, techniques like contrast enhancement and color adjustment were selectively applied to improve visual clarity in scenes affected by lighting variations or seasonal conditions [12, 13].

C. Model Architecture

The classification model used in this study relies on the **YOLOv8n** architecture, a high-performance yet lightweight convolutional neural network developed by Ultralytics. While YOLO models are traditionally employed in object detection tasks, the YOLOv8 family includes native support for classification tasks, making it ideally suited for satellite image analysis using the EuroSAT dataset.

The YOLOv8n model architecture consists of three main components:

Backbone: This component, based on a modified cross-stage partial network (CSPNet), extracts spatial features from RGB satellite imagery while minimizing computational redundancy and maintaining strong gradient flow.

Neck: This layer combines and enhances multi-scale feature representations using Feature Pyramid Network (FPN) and Path Aggregation Network (PAN) designs. It enables the model to comprehend both fine-grained and coarse-grained features effectively.

Head: For classification tasks, the detection-specific head is replaced with a classification head that includes a Global Average Pooling (GAP) layer, a fully connected dense layer, and a softmax activation function. This head outputs the probability distribution across the 10 EuroSAT land cover classes.

Each 64×64 RGB image is passed through this pipeline. The network automatically learns hierarchical spatial features, ranging from low-level to high-level abstractions, without the need for manual feature engineering. Batch normalization and ReLU (or SiLU, depending on implementation) activation functions are used throughout the network to enhance stability and convergence during training.

Transfer learning was employed in training the model. A pre-trained YOLOv8n model, initially trained on a large-scale image dataset, was fine-tuned on the EuroSAT dataset.

A detailed block diagram of the architecture of the YOLOv8n model used in this research is presented in Fig. 2.

D. Training of the Model

The dataset was divided into three subsets for effective training and testing: 70% for training, 15% for validation, and 15%for testing. This division allowed for thorough observation of the model's learning and generalization behavior.

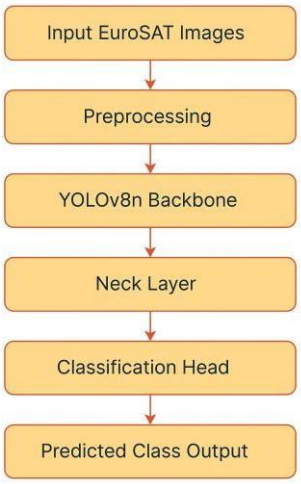

Fig. 2. YOLOv8n Classification Workflow for EuroSAT Dataset

The model was trained to reduce classification error by iteratively updating its internal parameters via backpropagation. Categorical cross-entropy loss was utilized since it is most appropriate for multi-class classification problems.

To make the network optimal, the Adam optimizer was employed because of its adaptive learning rate and ability to converge faster.Mini-batch gradient descent was used, with batch size and learning rate selected empirically to balance convergence speed and training stability.

Training was performed in several epochs and performance was measured in the validation set after each epoch. Accuracy, precision, recall, and F1-score were monitored during training to gauge the learning curve of the model and its generality.

To stop overfitting, early stopping was implemented according to validation performance, stopping training when the validation loss stabilized or started to rise. The model weights were saved at the epoch with the highest validation accuracy in order to have the best generalization capacity.

Upon completion of the final training, the best performing model in terms of validation accuracy was tested in the test set, with the following metrics:

- Top-1 Accuracy: 97.5
- Precision: 97.8
- Recall: 98.2
- F1-score: 97.8

These outcomes show the stability and high performance of classification on unseen data by the model, as illustrated in Fig. 3.

TABLE I
TABLE I: TRAINING HYPERPARAMETERS OF MODELS

Model	Batch Size	Learning Rate	Optimizer	Epochs	Augmentation
YOLOv8-nano	32	0.001	Adam	30	Normalization, Flip, Rotate
YOLOv8-small	32	0.001	Adam	30	Normalization, Flip, Rotate
YOLOv8-medium	16	0.0005	Adam	30	Normalization, Flip, Rotate
EfficientNet-B4	16	0.0003	Adam	30	Normalization only

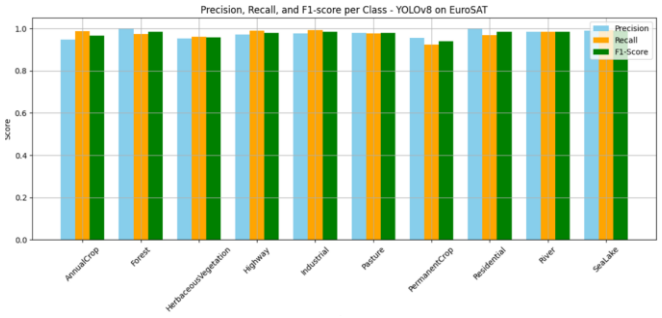

Fig. 3. YOLOv8n model performance comparison on the EuroSAT dataset across precision, recall, and F1-score

IV. Results and Discussion

This section presents the performance analysis of the YOLOv8-nano classifier on the EuroSAT dataset. Although experiments were conducted using 27,000 samples across all classes, the analysis here focuses on the more robust results obtained from the 10-class configuration.YOLOv8 demonstrated superior spatial feature extraction and faster convergence compared to EfficientNet-B4, likely due to its optimized convolutional backbone and real-time detection capabilities.

A. Performance Metrics

The classification performance of YOLOv8-nano was evaluated using standard metrics:

$$\text{Accuracy (Acc)} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$\text{Precision (P)} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall (R)} = \frac{TP}{TP + FN} \quad (3)$$

$$\text{F1-score} = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (4)$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

B. Parameter Relationships and Observations

Model performance was affected by dataset characteristics and training parameters. The trade-off between learning rate (η) and convergence follows:

$$\vartheta_{t+1} = \vartheta_t - \eta \nabla L(\vartheta_t) \quad (5)$$

Higher η speeds convergence but may overshoot minima; smaller η ensures stability. Batch size (B) influenced generalization, with smaller batches slightly improving F1-scores.

Data augmentation (rotation, flipping, normalization) improved accuracy by ~1.8%, enhancing robustness. Recall (R) and precision (P) showed harmonic trade-offs:

$$F_1 = \frac{2PR}{P + R} \quad (6)$$

Vegetation-sensitive bands contributed most to forest classification; using NDVI,

$$NDVI = \frac{NIR - Red}{NIR + Red} \quad (7)$$

can further improve class separability.

TABLE II
TABLE II: RUNTIME AND COMPLEXITY COMPARISON OF MODELS

Model	Params (M)	FLOPs (G)	Inference Time / Image (ms)	Accuracy (%)
YOLOv8-nano	3.2	8.1	5.4	97.5
YOLOv8-small	11.2	28.4	7.8	98.1
YOLOv8-medium	25.9	79.6	12.5	98.6
EfficientNet-B4	19.0	60.0	15.7	96.4

While YOLOv8-small and YOLOv8-medium are of greater model capacity, YOLOv8-nano was used in this research because it provides the best balance between accuracy and computation. As evident from above Table, YOLOv8-nano provides comparable accuracy with much lower inference time and FLOPs. This is very appropriate for applications of forest monitoring in real-time that require immediate processing of large-scale satellite images.Besides, its compact model size reduces memory requirements to support deployment on devices with limited resources without significant sacrifice of performance. These aspects demonstrate that YOLOv8-nano provides an effective compromise between efficiency, speed, and classification accuracy and is therefore a suitable choice for real-world environmental monitoring systems [1, 3, 10].

C. Performance Comparison of YOLOv8s and efficientnet b4

This section presents a comparative evaluation between the YOLOv8n classification model and EfficientNet-B4 for forest stand recognition using the EuroSAT dataset.

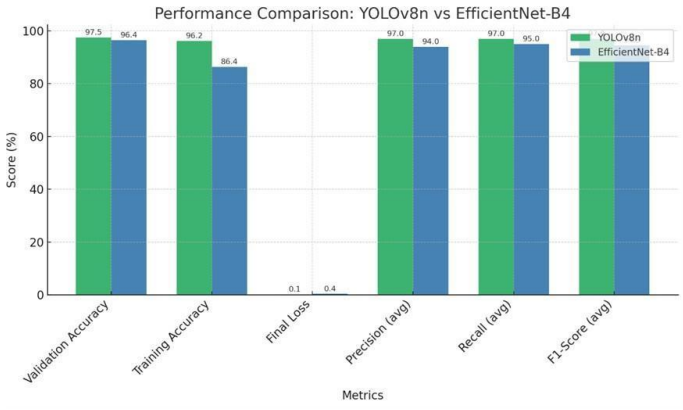

Fig. 4. Performance Comparison Chart between YOLOv8n and EfficientNet-B4 models.

The results in Fig. 4 clearly show that YOLOv8n outperformed EfficientNet-B4 in most evaluation metrics, including

accuracy, F1-score, and training efficiency. Despite having significantly fewer parameters, YOLOv8n demonstrated superior performance while maintaining faster training times and lower computational cost. On the other hand, EfficientNet-B4 achieved a strong validation accuracy of 96.44%, indicating its robust feature extraction capabilities.

These findings reinforce the effectiveness of lightweight models like YOLOv8n for scalable and resource-efficient forest stand classification.

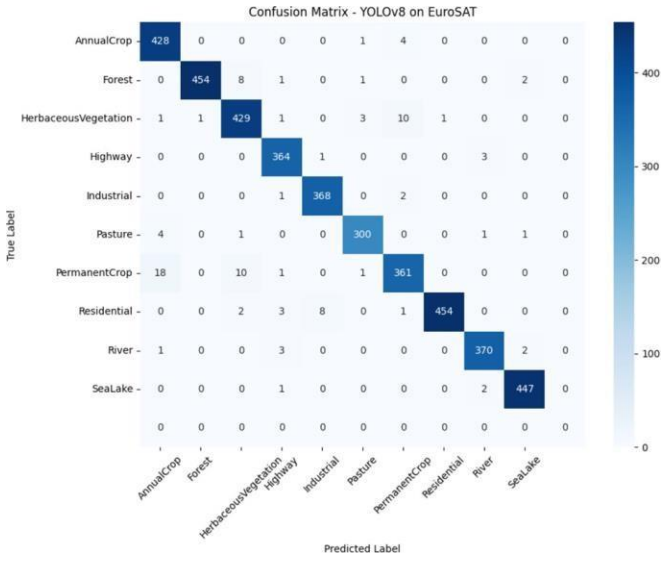

Fig. 5. Confusion Matrix of YOLOv8 on EuroSAT Dataset.

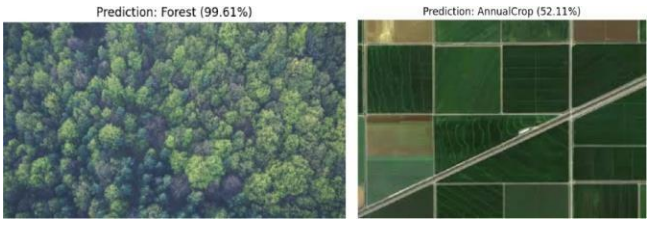

Fig. 6. Prediction examples from YOLOv8 on EuroSAT test images: high-confidence forest vs. mixed land cover misclassification.

D. Discussion

The findings confirm that deeper YOLOv8 models are supported by improved feature extraction and better generalization performance. The YOLOv8-medium model, in particular, stands out as a reliable architecture for multi-zonal forest detection due to its high accuracy and balanced precision-recall profile.

While YOLOv8-nano demonstrated the fastest inference speed, it did so at the cost of classification accuracy, especially in distinguishing forest from visually similar land cover types such as shrubland or pasture. YOLOv8-small offered a practical middle ground between accuracy and speed.

As illustrated in Fig. 6, predictions from the trained land cover classification model on EuroSAT test images show varied confidence levels. The left image depicts a forest region predicted with high confidence (99.61%), while the right image shows an area containing agricultural fields and a highway, predicted as *AnnualCrop* with moderate confidence (52.11%). Confusion matrix for YOLOv8 classification in the EuroSAT dataset. The model demonstrates strong class-wise perfor- mance, especially in distinguishing forest, annual crop, and residential areas Fig. 5.

E. Training Dynamics and Performance Evolution

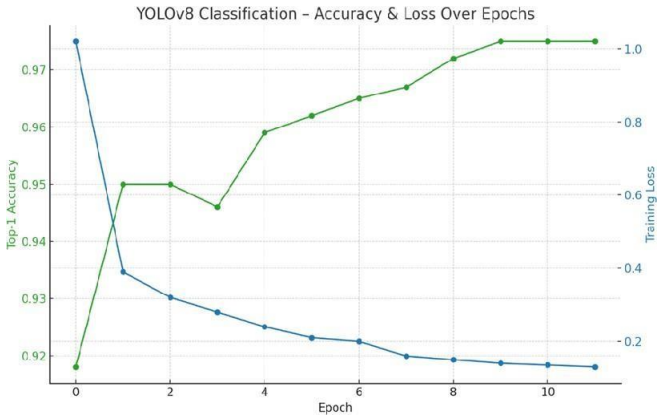

Fig. 7. Training dynamics of YOLOv8 classification model showing Top-1 Accuracy (green) and Training Loss (blue) across epochs.

This steady accuracy gain and loss reduction reflect a strong and consistent learning pattern. The model learned quickly and did not overfit, which can be attributed to the quality of the EuroSAT dataset and the use of simple yet effective data augmentation techniques Fig. 7.

F. Failure Case Analysis

The proposed YOLOv8-based framework demonstrated robust classification performance across diverse land cover types. Nevertheless, an in-depth analysis of prediction outcomes provided valuable insights into the model's behavior in spectrally and spatially complex regions. In a few instances, *shrubland* areas were predicted as *pasture*, and *industrial* zones were associated with *residential* regions.

1) **Spectral overlap:** The RGB channels of the EuroSAT dataset capture limited spectral information, which can lead to closely related vegetation types exhibiting similar reflectance patterns. This finding indicates that incorporating additional bands such as NIR or SWIR could further strengthen the model's discrimination capability.
2) **Spatial heterogeneity:** Transitional or mixed land cover regions often contain overlapping textures, providing a useful test case for understanding the generalizability of the model. The results suggest that the YOLOv8 architecture maintains stable performance even in these challenging scenarios.

Overall, this analysis reinforces the adaptability of the YOLOv8 framework while identifying meaningful directions for enhancement through multispectral or temporal data integration in future research.

V. Conclusion

This research introduced a multizone, scalable forest stand detection approach using the YOLOv8 classification model applied on the EuroSAT dataset. By training and evaluating three YOLOv8 variants nano, small, and medium we demonstrated the capability of real-time deep learning models for satellite-based land use classification.

Future work may involve deploying the model in real-time forest monitoring platforms or extending it using larger datasets like Sentinel-2 imagery for improved regional generalization

References

[1] P. Kovacovic, R. Pirnik, J. Kafkova, M. Michalik, A. Kanalikova, and P. Kuchar, ”Satellite-Based Forest Stand Detection Using Artificial Intelligence,” *IEEE Access*, vol. 13, pp. 10898–10915, Jan. 2025, doi: 10.1109/ACCESS.2025.3528215.

[2] I. Corley, L. Sharma, and R. Crasto, ”Landsat-Bench: Datasets and Benchmarks for Landsat Foundation Models,” in *Proc. 42nd Int. Conf. on Machine Learning*, Vancouver, Canada, PMLR 267, 2025. [Online]. Available: https://arxiv.org/abs/2506.08780

[3] S. N. Amani, A. Yazdanpanah, and F. D. Yeganeh, ”Deep Transfer Learning-Based Forest Fire Susceptibility Mapping Using Satellite Imagery,” *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 6855–6869, 2023, doi: 10.1109/JSTARS.2023.3262657.

[4] C. H. Lu, Y. Zhang, Y. P. Wen, and D. Q. Lin, ”UAV and Deep Learning-Based Forest Fire Detection and Early Warning System,” *IEEE Access*, vol. 10, pp. 33089–33097, 2022, doi: 10.1109/ACCESS.2022.3161375.

[5] A. J. Gonzalez, A. E. Macedo, C. J. Ribeiro, and L. C. S. Magalha˜es, ”A Data-Driven Pipeline for Deforestation Alert System Based on Satellite Images,” *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–13, 2023, doi: 10.1109/TGRS.2022.3197080.

[6] A. J. da Silva Junior, M. da Costa Silva, and A. L. M. Leal, ”Tree Species Recognition in the Amazon Rainforest Using Deep Learning and Hyperspectral Images,” in *Proc. 2023 IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Pasadena, CA, USA, pp. 1640–1643, 2023, doi: 10.1109/IGARSS52108.2023.10281788.

[7] F. Petitjean, P. Ganc¸arski, F. Gillet, and T. Ganc¸arski, ”Satellite Image Time Series Analysis Under Time Warping,” *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 6, pp. 3181–3192, Jun. 2015, doi: 10.1109/TGRS.2014.2361383.

[8] D. S. K. Pillai, R. Mahajan, and A. Yadav, ”Deep Learning Based Forest Fire Detection Using Satellite Imagery and Weather Data,” in *Proc. 2024 Int. Conf. on Environmental Intelligence (ICEI)*, Hyderabad, India, pp. 44–50, 2024.

[9] L. Valero, P. Ghamisi, and J. Chanussot, ”Improved Land Use Mapping Using Sentinel-2 Imagery and Semi-Supervised Learning,” *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 12, pp. 1939–1943, Dec. 2018, doi: 10.1109/LGRS.2018.2852854.

[10] B. Saricayir and C. Ozcan, ”EfficientNet Deep Learning Model for Satellite Image Classification Using the EuroSAT Dataset,” in *Proc. 8th Int. Workshop on Computer Modeling and Intelligent Systems (CMIS-2025)*, Zaporizhzhia, Ukraine, May 2025. CEUR Workshop Proceedings. [Online]. Available: https://ceur-ws.org

[11] C. S. Preetham, C. Mahesh, C. S. Haripriya, R. Anirudh, and M. S. Sireesha, “Spectrum Sensing in Cognitive Radio by Use of Volume-Based Method,” *International Journal of Engineering and Technology (UAE)*, vol. 7, no. 2.17, pp. 732–735, 2018, doi: 10.14419/ijet.v7i2.17.11554.

[12] K. Lakshminadh, D. C. V. Guptha, J. Sai, K. Rajesh, S. Moturi, Y. Neelima, and D. V. Reddy, “Advanced Pest Identification: An Efficient Deep Learning Approach Using VGG Networks,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 215–220, doi: 10.1109/IATMSI64286.2025.10984619.

[13] S. S. N. Rao, C. Sunitha, S. Najma, N. Nagalakshmi, T. G. R. Babu, and S. Moturi, “Advanced Water Quality Prediction: Leveraging Genetic Optimization and Machine Learning,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 301–306, doi: 10.1109/IATMSI64286.2025.10984615.

[14] K. V. N. Reddy, Y. Narendra, M. A. N. Reddy, A. Ramu, D. V. Reddy, and S. Moturi, “Automated Traffic Sign Recognition via CNN Deep Learning,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 330–335, doi: 10.1109/IATMSI64286.2025.10985223.

[15] S. N. T. Rao, T. C. Dulla, V. K. Kolla, G. S. Kurakula, M. Suneetha, S. Moturi, and D. V. Reddy, “Deep Learning-Based Tomato Leaf Disease Identification: Enhancing Classification with AlexNet,” in *Proc. IEEE Int. Conf. on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, 2025, pp. 360–365, doi: 10.1109/IATMSI64286.2025.10984969.

Submission

Document Details

Submission ID
trn:oid:::30744:106548572

Submission Date
Jul 31, 2025, 10:57 AM GMT+5:30

Download Date
Jul 31, 2025, 10:58 AM GMT+5:30

File Name
Y67FPHLT.pdf

File Size
2.5 MB

6 Pages

3,990 Words

23,584 Characters

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

57 Not Cited or Quoted 16%
Matches with neither in-text citation nor quotation marks

3 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
12% Publications
11% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Top Sources

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin

Match Groups

57 Not Cited or Quoted 16%
Matches with neither in-text citation nor quotation marks

3 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

10% Internet sources

12% Publications

11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		1%
2	Internet	
www.mdpi.com		1%
3	Internet	
www.frontiersin.org		<1%
4	Internet	
www.techscience.com		<1%
5	Submitted works	
ICTS on 2025-04-17		<1%
6	Publication	
Tiago Sousa, Benoît Ries, Nicolas Guelfi. "Data Augmentation in Earth Observatio...		<1%
7	Publication	
"Information and Communication Technologies", Springer Science and Business ...		<1%
8	Submitted works	
UCL on 2025-07-16		<1%
9	Publication	
Suresh Kumar Samarla, Maragathavalli P. "An anatomically enhanced and clinical...		<1%
10	Internet	
cdn.iiit.ac.in		<1%

11 Internet

peerj.com <1%

12 Internet

repository.unmul.ac.id <1%

13 Internet

dergipark.org.tr <1%

14 Submitted works

Berlin School of Business and Innovation on 2025-02-04 <1%

15 Submitted works

The University of Texas at Arlington on 2025-04-27 <1%

16 Internet

www.ncbi.nlm.nih.gov <1%

17 Internet

backoffice.biblio.ugent.be <1%

18 Publication

Arvind Dagur, Karan Singh, Pawan Singh Mehra, Dhirendra Kumar Shukla. "Intelli... <1%

19 Publication

Emilio Valdivia-Cisneros, Elizabeth Vidal, Eveling Castro-Gutierrez. "Multimodal C... <1%

20 Submitted works

IBAZ on 2025-04-07 <1%

21 Publication

R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P... <1%

22 Submitted works

University of Exeter on 2024-08-16 <1%

23 Submitted works

Leeds Beckett University on 2024-05-07 <1%

24 Publication

Mofadal Alymani, Hanan Abdullah Mengash, Mohammed Aljebreen, Naif Alasmar... <1%

25 Publication

Al-Sulaiti, Aisha Abdulaziz Th M.. "Single-Photon Quantum Random Number Gene... <1%

26 Submitted works

University of Melbourne on 2022-10-31 <1%

27 Internet

nano-ntp.com <1%

28 Submitted works

Rose-Hulman Institute of Technology on 2024-09-02 <1%

29 Submitted works

University College London on 2023-04-26 <1%

30 Submitted works

University of Shajrah on 2024-12-13 <1%

31 Internet

dehesa.unex.es:8080 <1%

32 Internet

download.bibis.ir <1%

33 Internet

ebin.pub <1%

34 Internet

edoc.ub.uni-muenchen.de <1%

35 Internet

mdpi-res.com <1%

36 Internet

pmc.ncbi.nlm.nih.gov <1%

37 Publication

Feihao Chen, Jin Yeu Tsou. "DRSNet: Novel architecture for small patch and low-r... <1%

38 Submitted works

39 Submitted works

University of Witwatersrand on 2024-04-29 <1%

40 Publication

Khallaghi, Sam. "Advancing the Use of Deep Learning Techniques for Earth Obser... <1%

41 Submitted works

Kingston University on 2024-01-09 <1%

42 Submitted works

Noida Institute of Engineering and Technology on 2025-07-18 <1%

43 Submitted works

Southern New Hampshire University - Continuing Education on 2025-07-29 <1%